



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

编译原理上机作业调研报告

语言处理系统完整流程与 SysY 等价 LLVM
IR/RISC-V 程序实验

成员：林子媛 (2410936)

成员：王麒萱 (2412096)

班级：信息安全班

2026 年 10 月

摘要

这次编译原理上机实验由成员 A 和成员 B 分工完成。小题 1 中，成员 A 以带有普通宏、带参宏和条件编译的 C++ 阶乘程序为例，使用 GCC 和 Clang 观察预处理、词法分析、tree、CFG、RTL、汇编、目标文件和链接后的可执行文件；成员 B 以包含宏、数组和循环的 C17 程序为例，使用 Clang/LLVM 保存预处理结果、token、AST、CFG、LLVM IR、AArch64 汇编以及 ELF 符号和重定位信息，并比较 -O0 与 -O2 的输出差异。两部分的命令、截图和中间文件均保留在仓库中。

小题 2 中，两人共同设计了一个 SysY 程序：读入整型数组，计算偶数元素之和，并统计正数元素个数。成员 A 编写了等价的 LLVM IR，成员 B 编写了等价的 RISC-V 汇编，两种实现都链接 SysY 运行时库完成验证。LLVM IR 分别在本机程序和 lli 中测试，RISC-V 程序在 QEMU 中测试；三组统一数据的结果均与预期一致，RISC-V 版本补充的 100 组特殊和普通数据也全部通过。实验结果表明，LLVM IR 可以用基本块和 SSA 值表示循环、分支和数组访问，而 RISC-V 汇编还需要直接处理寄存器、栈帧和数组地址。

进阶部分以 2×4 、 4×3 矩阵乘法观察 MLIR 中端处理。成员 A 整理了 linalg.matmul 到三层 scf.for 的表示变化，成员 B 核对基本块形式、LLVM dialect、LLVM IR 和 x86-64 目标文件。保存的 matmul.ll 能重新编译为导出 matmul 符号的 ELF 可重定位文件。

关键词：GCC；编译流程；SysY；LLVM IR；RISC-V；运行时库

成员分工

小题 1 中，成员 A 使用 C++ 阶乘程序进行 GCC 编译流程实验，成员 B 使用 C 程序进行 Clang/LLVM 编译流程实验。小题 2 中，我们共同编写 SysY 程序并确定测试数据，成员 A 编写 LLVM IR，成员 B 编写 RISC-V 汇编。进阶部分以同一份矩阵乘法 MLIR 程序为对象，成员 A 整理高层表示和循环 Lowering，成员 B 核对控制流、LLVM IR 与目标文件。报告中的摘要、引言、结论、参考文献和整体排版由两人共同整理。

表 1 小组任务分工

内容	成员 A	成员 B
小题 1	C++ 阶乘程序、GCC 编译流程分析及对应内容撰写	C 程序、Clang/LLVM 编译流程分析及对应内容撰写
SysY 程序	共同编写程序并确定三组基础测试数据	
LLVM IR	编写 LLVM IR，完成编译、链接及 native、lli 测试	共同核对测试结果
RISC-V	共同核对测试结果	编写 RISC-V 汇编，完成汇编、运行时库链接、QEMU 测试及扩展测试
MLIR 进阶	整理 arith、linalg.matmul 及其循环 Lowering 结果	核对控制流、LLVM IR 和 x86-64 目标文件，验证 matmul 符号
报告整理	摘要、引言、结论、参考文献及报告整体排版共同完成	

目录

成员分工	1
一、 引言	1
二、 实验环境与总体方法	1
三、 小题 1: 编译过程分析	1
(一) 成员 A: C++/GCC 宏与编译流程实验	1
1. 实验环境与源程序	1
2. 预处理器	4
3. 编译器内部阶段	8
4. 汇编器	15
5. 链接器	17
6. 编译参数扩展探索	20
7. 本小题小结	22
(二) 成员 B: Clang/LLVM 编译流程实验	22
1. 源程序与预处理	22
2. 词法、语法与语义分析证据	24
3. 控制流图与 LLVM IR	26
4. 优化前后比较与目标代码生成	28
5. 链接、反汇编与运行验证	30
6. 阶段观察小结	31
四、 小题 2: LLVM IR 与 RISC-V 汇编编程	31
(一) SysY 示例程序与测试数据	31
(二) LLVM IR 实现	33
1. SysY 与 LLVM IR 的对应	33
2. LLVM IR 程序	34
3. 编译、链接与运行	36
4. 测试结果	37
5. 本部分小结	37
(三) RISC-V 汇编程序	37
1. 寄存器与主要指令	37
2. RISC-V 汇编代码	38
3. 汇编与链接	40
4. 运行结果	41
5. 符号检查	43
6. 本部分小结	43
五、 进阶要求: 观察 AI 编译器 MLIR 中端处理过程	43
(一) 任务与分工	44
(二) 基础 MLIR 与 Canonicalization	44
(三) Linalg Dialect 中的矩阵乘法	45

（四） Linalg 到 SCF、MemRef 和 Arith 的 Lowering	46
（五） SCF 到控制流形式的 Lowering	48
（六） 转换为 LLVM Dialect	50
（七） 标准 LLVM IR 与目标文件验证	52
（八） 本部分小结	54
六、 结论	55

一、 引言

我们把这次作业当作一次从源程序到可执行文件的追踪练习。只看最后的运行结果，很难知道宏、类型检查、控制流、符号和库函数分别在哪个阶段发生变化。因此，小题 1 保留了预处理文件、token、tree、CFG、RTL、汇编、目标文件和链接结果，再结合源码逐项说明它们的作用。

小题 2 不再只观察编译器生成的内容，而是直接写出同一份 SysY 程序的 LLVM IR 和 RISC-V 汇编。两种实现使用相同的函数划分和测试输入，通过运行结果检查数组访问、循环、分支、函数调用和运行时接口是否保持一致。

二、 实验环境与总体方法

成员 A 的小题 1 实验在 WSL2 Ubuntu 22.04 环境中完成，主要使用 GCC 11.4.0，并在词法分析阶段使用 Clang 14 输出 token 序列，目标平台为 x86-64。成员 B 的小题 1 实验在 Ubuntu 22.04 Docker 容器中完成，主要使用 Clang/LLVM 14.0.0、GCC 11.4.0、Graphviz 2.43.0；流程实验另观察 AArch64 目标汇编与 ELF 文件。成员 A 的 LLVM IR 实验以 x86-64 Linux 为运行目标，在 Ubuntu 22.04 Docker 容器中完成。RISC-V 部分使用课程提供的交叉工具链与 QEMU，目标为 RV64，并链接课程提供的 SysY RISC-V 运行时库。

实验分两条线进行。第一条线使用 C/C++ 程序，保存预处理文件、token、GCC tree、CFG、RTL、汇编、目标文件和可执行文件，并比较不同编译参数带来的变化。第二条线以共享 SysY 程序为准，分别完成 LLVM IR 和 RISC-V 版本，再通过汇编、链接和多组输入检查两种实现的输出。

三、 小题 1: 编译过程分析

按照实验要求，小题 1 以简单的 C/C++ 程序为例，分别观察预处理、编译、汇编和链接各阶段的结果，并结合不同编译参数进行比较。成员 A 使用带有普通宏、带参宏和条件编译的 C++ 阶乘程序，使用 GCC 分析各阶段输出；成员 B 使用包含宏、数组、循环和条件分支的 C 程序，使用 Clang/LLVM 分析预处理结果、token、AST、CFG、LLVM IR、汇编和目标文件。两部分实验最后都通过实际输入运行程序，并检查链接后的执行结果。

（一） 成员 A: C++/GCC 宏与编译流程实验

本部分由成员 A 负责。实验使用一个 C++ 阶乘程序，按照 GCC 的预处理、编译、汇编和链接流程依次生成各阶段结果。源程序中加入自定义头文件、普通宏、带参宏、条件编译、全局变量、全局常量、函数、条件分支和循环，用于观察这些内容经过各编译阶段后的变化。实验环境为 WSL2 下的 Ubuntu 22.04，g++ 版本为 11.4.0，目标平台为 x86-64，中间文件统一保存在 output 目录中。

1. 实验环境与源程序

自定义头文件 test.h 定义输入上限 N、带参宏 mul(a,b) 和条件编译宏 DEBUG，具体内容如图 1 所示。

```
C test.h
1  #ifndef TEST_H
2  #define TEST_H
3
4  //宏
5  #define N 20
6
7  //带参宏
8  #define mul(a,b) ((a)*(b))
9
10 //条件编译
11 #ifdef DEBUG
12 #define debug(x) cout << x << endl
13 #else
14 #define debug(x)
15 #endif
16
17 #endif
```

图 1 自定义头文件 test.h

main.cpp 中包含全局变量 times、全局常量 start、阶乘函数 factorial(int n) 和主函数，源程序如图 2 所示。

```
main.cpp
1  //头文件
2  #include <iostream>
3  //自定义头文件
4  #include "test.h"
5  using namespace std;
6
7  //全局变量
8  int times = 0;
9  //全局常量
10 const int start = 2;
11
12 //函数
13 long long factorial(int n)
14 {
15     //局部变量
16     long long result = 1;
17
18     //循环
19     for (int i = start; i <= n; i++)
20     {
21         //运算、赋值、宏
22         result = mul(result, i);
23         times++;
24     }
25     return result;
26 }
27
28 int main()
29 {
30     int n;
31
32     //输入
33     cin >> n;
34     //条件、逻辑运算
35     if (n < 0 || n > N)
36     {
37         return 0;
38     }
39     //条件编译
40     debug(n);
41     //函数调用、输出
42     cout << factorial(n) << endl;
43     cout << times << endl;
44
45     return 0;
46 }
```

图 2 主程序 main.cpp

实验目录和 GCC 版本如图 3 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ pwd
ls
g++ --version
/mnt/d/compilerLab
image main.cpp output test.h
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 3 实验目录与 GCC 版本

首先直接编译并运行源程序:

```
1 g++ main.cpp -o main
2 ./main
```

输入 5 后, 程序输出 120 和 1。其中 120 为 $5!$ 的结果, 1 为 factorial 函数的调用次数, 运行结果如图 4 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ main.cpp -o main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./main
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 4 基准程序运行结果

2. 预处理器

使用 `-E` 选项只执行预处理, 并将结果保存为 `main.ii`:

```
1 g++ -E main.cpp -o output/main.ii
```

生成结果如图 5 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -E main.cpp -o output/main.ii
i
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls output
main.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 5 生成预处理文件 `main.ii`

预处理阶段会处理 `#include`。在 `main.ii` 中可以看到 `iostream` 的内容已经展开, 同时保留了 `test.h` 的文件切换标记, 结果如图 6 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "iostream" output/main.i
i | head
8:# 1 "/usr/include/c++/11/iostream" 1 3
9:# 36 "/usr/include/c++/11/iostream" 3
11:# 37 "/usr/include/c++/11/iostream" 3
73:# 39 "/usr/include/c++/11/iostream" 2 3
1217:     class basic_istream;
1272: typedef basic_istream<char> istream;
1312: typedef basic_istream<wchar_t> wistream;
25106: const error_category& istream_category() noexcept;
25110: { return error_code(static_cast<int>(__e), istream_category()); }
25114: { return error_condition(static_cast<int>(__e), istream_category()
); }
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "test.h" output/main.ii
| head
32255:# 1 "test.h" 1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 6 头文件展开后的内容

宏展开后, N 被替换为 20, `mul(result,i)` 被替换为 `((result)*(i))`, 源文件中的注释已经消失。没有定义 `DEBUG` 时, `debug(n)` 展开为空, 对应位置只剩一个分号, 如图 7 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ tail -n 50 output/main.ii

static ios_base::Init __ioinit;

}
# 3 "main.cpp" 2

# 1 "test.h" 1
# 5 "main.cpp" 2

# 5 "main.cpp"
using namespace std;

int times = 0;

const int start = 2;

long long factorial(int n)
{
    long long result = 1;

    for (int i = start; i <= n; i++)
        result = ((result)*(i));
    times++;
    return result;
}

int main()
{
    int n;

    cin >> n;

    if (n < 0 || n > 20)
        return 0;

    ;

    cout << factorial(n) << endl;
    cout << times << endl;

    return 0;
}
```

图 7 未定义 DEBUG 时的预处理结果

加入 -DDEBUG 后重新预处理:

```
1 g++ -E -DDEBUG main.cpp -o output/main_debug.ii
```

此时条件编译选择 DEBUG 分支, debug(n) 被展开为 cout << n << endl, 结果如图 8 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -E -DDEBUG main.cpp -o output/main_debug.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ tail -n 50 output/main_debug.ii

static ios_base::Init __ioinit;

}
# 3 "main.cpp" 2

# 1 "test.h" 1
# 5 "main.cpp" 2

# 5 "main.cpp"
using namespace std;

int times = 0;

const int start = 2;

long long factorial(int n)
{
    long long result = 1;

    for (int i = start; i <= n; i++)
        result = ((result)*(i));
    times++;
    return result;
}

int main()
{
    int n;

    cin >> n;

    if (n < 0 || n > 20)
        return 0;

    cout << n << endl;
```

图 8 定义 DEBUG 后的预处理结果

再比较源文件和预处理文件的规模。main.cpp 为 41 行，test.h 为 16 行，而 main.ii 达到 32296 行、约 761 KB。主要原因是 iostream 及其依赖的标准库头文件都被展开到了预处理结果中，具体结果如图 9 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ wc -l main.cpp test.h output/main.ii
  41 main.cpp
  16 test.h
32296 output/main.ii
32353 total
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh main.cpp test.h output/main.ii
-rwxrwxrwx 1 lenovo lenovo 695 Sep 22 09:11 main.cpp
-rwxrwxrwx 1 lenovo lenovo 761K Sep 22 10:03 output/main.ii
-rwxrwxrwx 1 lenovo lenovo 207 Sep 22 09:30 test.h
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 9 源文件与预处理文件规模比较

3. 编译器内部阶段

这一部分分别从词法分析、语法分析、语义分析、中间代码生成、代码优化和代码生成几个阶段分析编译结果。GCC 的 tree、CFG 和 RTL 用于观察主要的中间表示，词法分析部分使用 Clang 输出 token 序列作为补充。

词法分析 使用 Clang 输出 main.cpp 的 token 序列。为了减少标准头文件产生的大量输出，只保留来源于 main.cpp 的 token:

```
1 clang++ -Xclang -dump-tokens -fsyntax-only main.cpp 2>&1 \
2 | grep "main.cpp:" | head -n 60
```

结果中包含 using、int、const、for 等关键字，times、factorial 等标识符，以及数值常量和运算符。宏展开生成的 token 还保留了来自 test.h 的位置信息，如图 10 所示。

```
lenovo@LAPTOP-GST08NH0:/mnt/d/compilerLab$ cd /mnt/d/compilerLab
clang++ -Xclang -dump-tokens -fsyntax-only main.cpp 2>&1 | grep "main.cpp:" | head -n 60
using 'using'      Loc=<main.cpp:5:1>
namespace 'namespace' [LeadingSpace] Loc=<main.cpp:5:7>
identifier 'std'     [LeadingSpace] Loc=<main.cpp:5:17>
semi ';'            Loc=<main.cpp:5:20>
int 'int'           [StartOfLine] Loc=<main.cpp:8:1>
identifier 'times'   [LeadingSpace] Loc=<main.cpp:8:5>
equal '='           [LeadingSpace] Loc=<main.cpp:8:11>
numeric_constant '0' [LeadingSpace] Loc=<main.cpp:8:13>
semi ';'            Loc=<main.cpp:8:14>
const 'const'       [StartOfLine] Loc=<main.cpp:10:1>
int 'int'           [LeadingSpace] Loc=<main.cpp:10:7>
identifier 'start'   [LeadingSpace] Loc=<main.cpp:10:11>
equal '='           [LeadingSpace] Loc=<main.cpp:10:17>
numeric_constant '2' [LeadingSpace] Loc=<main.cpp:10:19>
semi ';'            Loc=<main.cpp:10:20>
long 'long'         [StartOfLine] Loc=<main.cpp:13:1>
long 'long'         [LeadingSpace] Loc=<main.cpp:13:6>
identifier 'factorial' [LeadingSpace] Loc=<main.cpp:13:11>
l_paren '('          Loc=<main.cpp:13:20>
int 'int'            Loc=<main.cpp:13:21>
identifier 'n'       [LeadingSpace] Loc=<main.cpp:13:25>
r_paren ')'          Loc=<main.cpp:13:26>
l_brace '{'          [StartOfLine] Loc=<main.cpp:14:1>
long 'long'          [StartOfLine] [LeadingSpace] Loc=<main.cpp:16:5>
long 'long'          [LeadingSpace] Loc=<main.cpp:16:10>
identifier 'result'   [LeadingSpace] Loc=<main.cpp:16:15>
equal '='             [LeadingSpace] Loc=<main.cpp:16:22>
numeric_constant '1' [LeadingSpace] Loc=<main.cpp:16:24>
semi ';'              Loc=<main.cpp:16:25>
for 'for'             [StartOfLine] [LeadingSpace] Loc=<main.cpp:19:5>
l_paren '('           [LeadingSpace] Loc=<main.cpp:19:9>
int 'int'             Loc=<main.cpp:19:10>
identifier 'i'        [LeadingSpace] Loc=<main.cpp:19:14>
equal '='             [LeadingSpace] Loc=<main.cpp:19:16>
identifier 'start'     [LeadingSpace] Loc=<main.cpp:19:18>
semi ';'              Loc=<main.cpp:19:23>
identifier 'i'         [LeadingSpace] Loc=<main.cpp:19:25>
lessequal '<='         [LeadingSpace] Loc=<main.cpp:19:27>
identifier 'n'         [LeadingSpace] Loc=<main.cpp:19:30>
semi ';'              Loc=<main.cpp:19:31>
identifier 'i'         [LeadingSpace] Loc=<main.cpp:19:33>
plusplus '++'          Loc=<main.cpp:19:34>
r_paren ')'            Loc=<main.cpp:19:36>
identifier 'result'     [StartOfLine] [LeadingSpace] Loc=<main.cpp:21:9>
equal '='               [LeadingSpace] Loc=<main.cpp:21:16>
l_paren '('             [LeadingSpace] Loc=<main.cpp:21:18 <Spelling=../test.h:8:18>
l_paren '('             Loc=<main.cpp:21:18 <Spelling=../test.h:8:19>
```

图 10 Clang 输出的 main.cpp 词法 token 序列

语法分析 使用 -fdump-tree-original-raw 生成 GCC 的 tree:

```
1 g++ -fdump-tree-original-raw -c main.cpp -o output/main_ast.o
```

生成的 tree 文件如图 11 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -fdump-tree-original-raw -c main.cpp -o output/main_ast.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls
image main main.cpp output test.h
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls output
main.ii main_ast.cpp.005t.original main_ast.o main_debug.ii
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 11 生成 GCC tree 输出文件

factorial 函数的 tree 中可以看到 function_decl、parm_decl、var_decl、identifier_node、statement_list 和 return_expr 等节点。函数、参数、局部变量和语句已经按照树结构组织，相关片段如图 12 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '61252,61340p' output/main_ast.cpp.005t.original
;; Function long long int factorial(int) (null)
;; enabled by -tree-original

@1      bind_expr      type: @2      vars: @3      body: @4
@2      void_type      name: @5      algn: 8
@3      var_decl       name: @6      type: @7      scpe: @8
                        srcp: main.cpp:16      init: @9
                        size: @10      algn: 64      used: 1
@4      statement_list 0 : @11      1 : @12      2 : @13
                        3 : @14
@5      type_decl      name: @15      type: @2      srcp: <built-in>:0
                        note: artificial
@6      identifier_node strg: result lngt: 6
@7      integer_type   name: @16      size: @10      algn: 64
                        prec: 64      sign: signed min : @17
                        max : @18
@8      function_decl  name: @19      type: @20      scpe: @21
                        srcp: main.cpp:13      args: @22
                        link: extern
@9      integer_cst    type: @7      int: 1
@10     integer_cst    type: @23      int: 64
@11     cleanup_point_expr type: @2      op 0: @24
@12     bind_expr      type: @2      vars: @25      body: @26
@13     cleanup_point_expr type: @2      op 0: @27
@14     return_expr    type: @2      expr: @28
@15     identifier_node strg: void lngt: 4
@16     type_decl      name: @29      type: @7      srcp: <built-in>:0
                        note: artificial
@17     integer_cst    type: @7      int: -9223372036854775808
@18     integer_cst    type: @7      int: 9223372036854775807
@19     identifier_node strg: factorial lngt: 9
@20     function_type   size: @30      algn: 8      retn: @7
                        prms: @31
@21     translation_unit_decl name: @32
@22     parm_decl      name: @33      type: @34      scpe: @8
                        srcp: main.cpp:13      argt: @34
                        size: @35      algn: 32      used: 1
@23     integer_type   name: @36      size: @37      algn: 128
                        prec: 128      sign: unsigned min : @38
                        max : @39
@24     decl_expr      type: @2
@25     var_decl       name: @40      type: @34      scpe: @8
                        srcp: main.cpp:19      init: @41
                        size: @35      algn: 32      used: 1
@26     statement_list 0 : @42      1 : @43
@27     expr_stmt      type: @2      line: 22      expr: @44
@28     init_expr      type: @7      op 0: @45      op 1: @3
@29     identifier_node strg: long long int lngt: 13
@30     integer_cst    type: @23      int: 8
@31     tree_list      valu: @34      chan: @46
```

图 12 factorial 函数的 GCC tree 片段

语义分析 复制源程序，并将一次 factorial(n) 调用改为 factorial("5")。修改后函数调用的语法形式没有改变，但实参类型由 int 变为 const char*，修改位置如图 13 所示。

```
//函数调用、输出
cout << factorial("5") << endl;
cout << times << endl;
```

图 13 故意修改的函数调用

编译时 GCC 报告 invalid conversion from 'const char*' to 'int', 同时指出 factorial(int) 的参数类型为 int。报错结果如图 14 所示, 可以看到 GCC 会根据函数声明和实际参数进行类型检查。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -c main_error.cpp -o output/main_error.o
main_error.cpp: In function 'int main()':
main_error.cpp:38:23: error: invalid conversion from 'const char*' to 'int' [-fpermissive]
 38 |     cout << factorial("5") << endl;
    |                      ^~~~
    |                      |
    |                  const char*
main_error.cpp:13:25: note: initializing argument 1 of 'long long int factorial(int)'
 13 | long long factorial(int n)
    |                      ~~~~
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 14 GCC 类型检查报错

中间代码生成 使用 -fdump-tree-all-graph 输出 GCC 多个 tree 阶段的 .dot 文件:

```
1 g++ -O0 -fdump-tree-all-graph -c main.cpp -o output/main_tree.o
```

生成的多阶段文件如图 15 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -fdump-tree-all-graph -c main.cpp -o output/main_tree.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ find . -maxdepth 2 -type f -name "*.dot" | sort | head -n 20
./output/main_tree.cpp.015t.cfg.dot
./output/main_tree.cpp.017t.ompexp.dot
./output/main_tree.cpp.018t.warn-printf.dot
./output/main_tree.cpp.022t.fixup_cfg1.dot
./output/main_tree.cpp.023t.ssa.dot
./output/main_tree.cpp.027t.fixup_cfg2.dot
./output/main_tree.cpp.028t.local-fnsummary1.dot
./output/main_tree.cpp.029t.einline.dot
./output/main_tree.cpp.051t.release_ssa.dot
./output/main_tree.cpp.052t.local-fnsummary2.dot
./output/main_tree.cpp.092t.fixup_cfg3.dot
./output/main_tree.cpp.093t.ehdisp.dot
./output/main_tree.cpp.097t.adjust_alignment.dot
./output/main_tree.cpp.233t.vecloer.dot
./output/main_tree.cpp.234t.cplxlower0.dot
./output/main_tree.cpp.236t.switchlower_00.dot
./output/main_tree.cpp.240t.ehcleanup2.dot
./output/main_tree.cpp.241t.resx.dot
./output/main_tree.cpp.243t.isel.dot
./output/main_tree.cpp.244t.optimized.dot
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 15 Tree/CFG 多阶段输出文件

将其中较早阶段的 cfg.dot 转换为图片后, factorial 中的 for 循环被分成初始化、条件判断、循环体、回跳和退出等基本块; main 中的输入、范围判断、函数调用和输出也被分成不同基本块, 结果如图 16 所示。

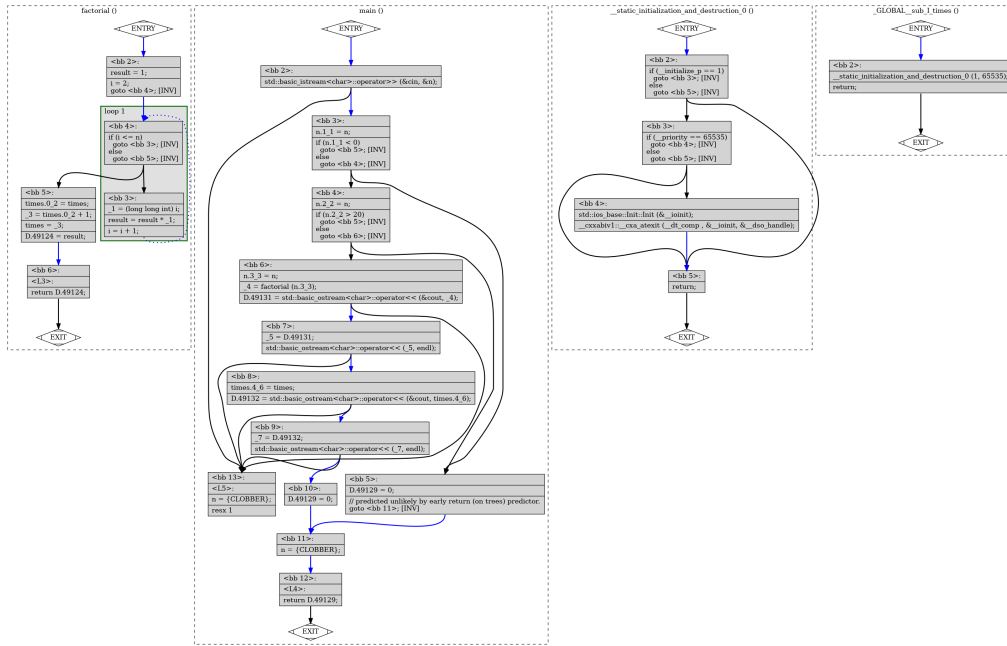


图 16 GCC 生成的 CFG

继续使用 `-fdump-rtl-all-graph` 输出 RTL:

```
1 g++ -O0 -fdump-rtl-all-graph -c main.cpp -o output/main_rtl.o
```

生成的 RTL 文件如图 17 所示, RTL expand 阶段的控制流程图如图 18 所示。RTL 中已经出现寄存器、函数调用、比较和跳转等底层内容。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -fdump-rtl-all-graph -c main.cpp -o output/main_rtl.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ find output -maxdepth 1 -type f -name "main_rtl*.dot" | sort | head -n 30
output/main_rtl.cpp.245r.expand.dot
output/main_rtl.cpp.246r.vregs.dot
output/main_rtl.cpp.247r.into_cfglayout.dot
output/main_rtl.cpp.248r.jump.dot
output/main_rtl.cpp.260r.reginfo.dot
output/main_rtl.cpp.284r.outof_cfglayout.dot
output/main_rtl.cpp.285r.split1.dot
output/main_rtl.cpp.287r.dfini.dot
output/main_rtl.cpp.288r.mode_sw.dot
output/main_rtl.cpp.289r.asmcons.dot
output/main_rtl.cpp.294r.lra.dot
output/main_rtl.cpp.295r.reload.dot
output/main_rtl.cpp.302r.pro_and_epilogue.dot
output/main_rtl.cpp.305r.jump2.dot
output/main_rtl.cpp.316r.split4.dot
output/main_rtl.cpp.317r.stack.dot
output/main_rtl.cpp.318r.zero_call_used_regs.dot
output/main_rtl.cpp.319r.alignments.dot
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 17 RTL 多阶段输出文件

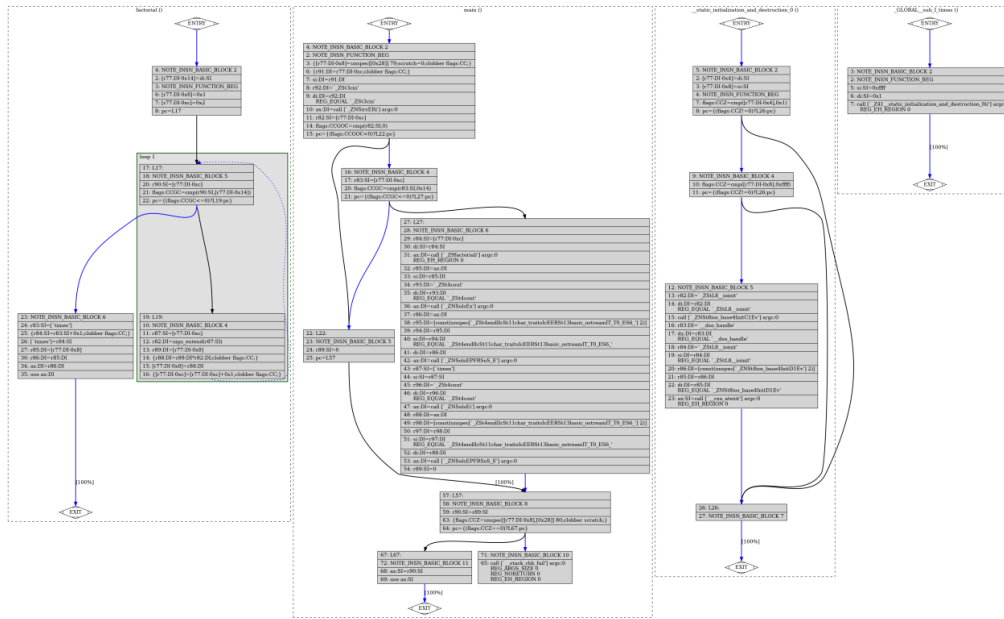


图 18 RTL expand 阶段控制流图

代码优化 使用同一个预处理文件分别生成 -O0 和 -O2 两个版本的汇编：

```
1 g++ -O0 -S -masm=att output/main.ii -o output/main_O0.s
2 g++ -O2 -S -masm=att output/main.ii -o output/main_O2.s
```

图 19 对比了两个版本的文件大小和行数。-O0 版本约 3.6 KB、193 行，-O2 版本约 4.2 KB、222 行，因此汇编文本变长并不代表优化效果变差。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O2 -S -masm=att output/main.ii -o output/main_O2.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_O0.s output/main_O2.s
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_O0.s
-rwxrwxrwx 1 lenovo lenovo 4.2K Sep 24 15:55 output/main_O2.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ wc -l output/main_O0.s output/main_O2.s
193 output/main_O0.s
222 output/main_O2.s
415 total
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_O2.s
73:      .globl _Z9factoriali
74:      .type _Z9factoriali, @function
75: _Z9factoriali:
103:     .size _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 19 -O0 与 -O2 汇编文件比较

再单独比较两个版本中的 factorial 函数。-O2 版本主要用寄存器保存循环变量和中间结果，循环体主要由 imulq、addq、cmpq 和 jne 等指令组成，times++ 也变为直接对内存加一。与 -O0 版本相比，-O2 版本减少了栈内存读写。图 20 为 -O2 下的 factorial 汇编。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '73,103p' output/main_O2
.s
        .globl _Z9factoriali
        .type _Z9factoriali, @function
_Z9factoriali:
.LFB1745:
        .cfi_startproc
        endbr64
        cmpl    $1, %edi
        jle     .L13
        leal    1(%rdi), %edx
        movl    $2, %eax
        movl    $1, %r8d
        .p2align 4,,10
        .p2align 3
.L12:
        imulq   %rax, %r8
        addq    $1, %rax
        cmpq    %rdx, %rax
        jne     .L12
        addl    $1, times(%rip)
        movq    %r8, %rax
        ret
        .p2align 4,,10
        .p2align 3
.L13:
        movl    $1, %r8d
        addl    $1, times(%rip)
        movq    %r8, %rax
        ret
        .cfi_endproc
.LFE1745:
        .size _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 20 -O2 下 factorial 函数的汇编结果

代码生成 使用 -s 让 GCC 在生成汇编后停止，得到 x86-64 AT&T 语法的 main_O0.s，如图 21 所示。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -S -masm=att output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00.s
-rwxrwxrwx 1 lenovo lenovo 761K Sep 22 10:03 output/main_00.s
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 21 由 main_00.s 生成汇编文件 main_O0.s

在 -O0 版本的 factorial 中，movq \$1,-8(%rbp) 对应 result=1，movl \$2,-12(%rbp) 对应循环初值，imulq 对应乘法，cmpl/jle 对应循环条件，返回前将 result 放入 %rax。相关汇编如图 22 所示。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '19,54p' output/main_00.s
s
        .globl _Z9factoriali
        .type   _Z9factoriali, @function
_Z9factoriali:
.LFB1731:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    %edi, -20(%rbp)
        movq    $1, -8(%rbp)
        movl    $2, -12(%rbp)
        jmp     .L2
.L3:
        movl    -12(%rbp), %eax
        cltq
        movq    -8(%rbp), %rdx
        imulq   %rdx, %rax
        movq    %rax, -8(%rbp)
        addl    $1, -12(%rbp)
.L2:
        movl    -12(%rbp), %eax
        cmpl    -20(%rbp), %eax
        jle     .L3
        movl    times(%rip), %eax
        addl    $1, %eax
        movl    %eax, times(%rip)
        movq    -8(%rbp), %rax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE1731:
        .size   _Z9factoriali, .-_Z9factoriali
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 22 -O0 下 factorial 函数的汇编结果

main 函数中包含 cin、cout 对应的库函数调用，以及 call _Z9factoriali。_Z9factoriali 是 C++ 对 factorial(int) 进行名字修饰后的符号名，相关代码如图 23 所示。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ sed -n '57,105p' output/main_00.s
main:
.LFB1732:
.cfi_startproc
endbr64
pushq   %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq    %rsp, %rbp
.cfi_def_cfa_register 6
subq    $16, %rsp
movq    %fs:40, %rax
movq    %rax, -8(%rbp)
xorl    %eax, %eax
leaq    -12(%rbp), %rax
movq    %rax, %rsi
leaq    _ZSt3cin(%rip), %rax
movq    %rax, %rdi
call    _ZNSirsEri@PLT
movl    -12(%rbp), %eax
testl   %eax, %eax
js      .L6
movl    -12(%rbp), %eax
cmpl    $20, %eax
jle     .L7
.L6:
movl    $0, %eax
jmp     .L9
.L7:
movl    -12(%rbp), %eax
movl    %eax, %edi
call    _Z9factoriali
movq    %rax, %rsi
leaq    _ZSt4cout(%rip), %rax
movq    %rax, %rdi
call    _ZNSolsEx@PLT
movq    _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6@GOTPCREL(%rip), %rdx
movq    %rdx, %rsi
movq    %rax, %rdi
call    _ZNSolsEPFRSoS_E@PLT
movl    times(%rip), %eax
movl    %eax, %esi
leaq    _ZSt4cout(%rip), %rax
movq    %rax, %rdi
call    _ZNSolsEi@PLT
movq    _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6@GOTPCREL(%rip), %rdx
movq    %rdx, %rsi
movq    %rax, %rdi
call    _ZNSolsEPFRSoS_E@PLT
movl    $0, %eax
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 23 main 函数中的输入、判断、函数调用和输出

4. 汇编器

使用 `-c` 将汇编文件转换为可重定位目标文件:

```
1 g++ -c output/main_00.s -o output/main_00.o
```

`file` 显示 `main_00.o` 为 ELF 64-bit LSB relocatable, x86-64。此时汇编指令已经编码为机器指令，但文件仍属于可重定位目标文件，结果如图 24 所示。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -c output/main_00.s -o output/main_00.o
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00.s output/main_00.o
-rwxrwxrwx 1 lenovo lenovo 3.4K Sep 24 16:07 output/main_00.o
-rwxrwxrwx 1 lenovo lenovo 3.6K Sep 24 15:45 output/main_00.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_00.o
output/main_00.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 24 生成可重定位目标文件 main_00.o

使用 `objdump -d` 对目标文件进行反汇编。图 25 中，左侧为十六进制机器码，右侧为对应的 x86 指令，例如 `eb 15` 对应 `jmp`，`7e e3` 对应 `jle`。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ objdump -d output/main_00.o > output/main_00_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00_disasm.s
-rwxrwxrwx 1 lenovo lenovo 5.6K Sep 24 16:09 output/main_00_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_00_disasm.s
7:0000000000000000 <_Z9factoriali>:
15: 1a:      eb 15          jmp     31 <_Z9factoriali+0x31>
24: 37:      7e e3          jle    1c <_Z9factoriali+0x1c>
25: 39:      8b 05 00 00 00 00 mov     0x0(%rip),%eax      # 3f
<_Z9factoriali+0x3f>
27: 42:      89 05 00 00 00 00 mov     %eax,0x0(%rip)      # 48
<_Z9factoriali+0x48>
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 25 目标文件 main_00.o 的反汇编结果

使用 `nm` 分析目标文件的符号表。图 26 中，`main` 和 `_Z9factoriali` 标记为 `T`，表示代码段中已经定义的全局符号；`times` 标记为 `B`，表示 BSS 段中的零初始化全局变量；`cin`、`cout` 等外部符号标记为 `U`，需要在链接阶段解析。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nm output/main_00.o > output/main_00_nm.txt
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -E "factorial|times|main" output/main_00_nm.txt
0000000000000015d t _GLOBAL__sub_I_times
0000000000000000 T _Z9factoriali
000000000000004e T main
0000000000000000 B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ head -n 30 output/main_00_nm.txt
t
      U _GLOBAL_OFFSET_TABLE_
0000000000000015d t _GLOBAL__sub_I_times
00000000000000107 t _Z41__static_initialization_and_destruction_0ii
0000000000000000 T _Z9factoriali
0000000000000000 r _ZL5start
      U _ZNSirsERi
      U _ZNSolsEPFRSoS_E
      U _ZNSolsEi
      U _ZNSolsEx
      U _ZNSt8ios_base4InitC1Ev
      U _ZNSt8ios_base4InitD1Ev
      U _ZSt3cin
      U _ZSt4cout
      U _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_
0000000000000004 b _ZStL8__ioint
      U __cxa_atexit
      U __dso_handle
      U __stack_chk_fail
000000000000004e T main
0000000000000000 B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 26 目标文件 main_00.o 的符号表

5. 链接器

使用 g++ 将 main_00.o 与程序所需的 C++ 运行库链接:

```
1 g++ output/main_00.o -o output/main_00
```

file 显示生成文件为 ELF 64-bit LSB PIE executable, 并采用动态链接。输入 5 后, 程序仍输出 120 和 1, 结果如图 27 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ output/main_00.o -o output/main_00
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_00

output/main_00: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=28f9861658df36d41be9b8b18706ef48911ed5b4, for GNU/Linux 3.2.0, not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./output/main_00
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 27 生成并运行动态链接可执行文件

对链接后的可执行文件再次进行反汇编。图 28 中, factorial 和 main 已经具有链接后的地址, main 中对 factorial 的调用也已经指向相应地址。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ objdump -d output/main_00 > output/main_00_exe_disasm.s
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "factorial" output/main_00_exe_disasm.s | head
169:0000000000001209 <_Z9factoriali>:
177:    1223:    eb 15                jmp     123a <_Z9factoriali+0x31>
186:    1240:    7e e3                jle     1225 <_Z9factoriali+0x1c>
218:    12a3:    e8 61 ff ff ff      call    1209 <_Z9factoriali>
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -n "<main>" output/main_00_exe_disasm.s | head
109:    1138:    48 8d 3d 18 01 00 00  lea     0x118(%rip),%rdi      # 1
257 <main>
194:0000000000001257 <main>:
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 28 链接后可执行文件的反汇编结果

最终可执行文件的符号表如图 29 所示。factorial、main 和 times 都已经具有确定地址。部分标准库符号仍显示为带版本信息的 U 项, 由动态链接器和共享库在程序装载或运行时解析。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nm output/main_00 > output/main_00_exe_nm.txt
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep -E "factorial|times|main"
output/main_00_exe_nm.txt
0000000000001366 t _GLOBAL__sub_I_times
0000000000001209 T _Z9factoriali
U __libc_start_main@GLIBC_2.34
0000000000001257 T main
000000000000427c B times
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ grep " U " output/main_00_exe_n
m.txt | head -n 20
U _ZNSirsERi@GLIBCXX_3.4
U _ZNSolsEPFRSoS_E@GLIBCXX_3.4
U _ZNSolsEi@GLIBCXX_3.4
U _ZNSolsEx@GLIBCXX_3.4
U _ZNSt8ios_base4InitC1Ev@GLIBCXX_3.4
U _ZNSt8ios_base4InitD1Ev@GLIBCXX_3.4
U _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_E
S6_@GLIBCXX_3.4
U __cxa_atexit@GLIBC_2.2.5
U __libc_start_main@GLIBC_2.34
U __stack_chk_fail@GLIBC_2.4
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 29 链接后可执行文件的符号表

使用 `-static` 生成静态链接版本，并与动态链接版本比较文件大小。动态链接版本约为 17 KB，静态链接版本约为 2.3 MB，结果如图 30 所示。静态链接版本包含更多库代码，因此文件大小明显增加。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -static output/main_00.o -o
output/main_static

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ file output/main_static
output/main_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linu
x), statically linked, BuildID[sha1]=f31cdc942fc7e8f7ded64835673bfc424bbc2
f3a, for GNU/Linux 3.2.0, not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_00 output/ma
in_static
-rwxrwxrwx 1 lenovo lenovo 17K Sep 24 17:01 output/main_00
-rwxrwxrwx 1 lenovo lenovo 2.3M Sep 24 19:39 output/main_static
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 30 动态链接与静态链接文件大小比较

静态链接版本输入 5 后同样输出 120 和 1，运行结果如图 31 所示。两种链接方式下程序的计算结果相同。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ./output/main_static
5
120
1
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 31 静态链接程序运行结果

6. 编译参数扩展探索

调试信息 `-g` 在相同的 `-O0` 条件下, 分别生成不带 `-g` 和带 `-g` 的可执行文件。图 32 中, `main_nog` 为 17 KB, `main_g` 为 34 KB。readelf 的结果显示, 不带 `-g` 的文件没有 `.debug` 段, 而带 `-g` 的文件包含 `.debug_info`、`.debug_line` 和 `.debug_str` 等调试信息。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 main.cpp -o output/main_nog
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ g++ -O0 -g main.cpp -o output/main_g
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ ls -lh output/main_nog output/main_g
-rwxrwxrwx 1 lenovo lenovo 34K Sep 24 20:40 output/main_g
-rwxrwxrwx 1 lenovo lenovo 17K Sep 24 20:40 output/main_nog
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ echo "=== no -g ==="
readelf -S output/main_nog | grep "\.debug" || echo "no debug sections"

echo "=== with -g ==="
readelf -S output/main_g | grep "\.debug"
=== no -g ===
no debug sections
=== with -g ===
[28] .debug_aranges PROGBITS 0000000000000000 0000303d
[29] .debug_info PROGBITS 0000000000000000 0000306d
[30] .debug_abbrev PROGBITS 0000000000000000 00005559
[31] .debug_line PROGBITS 0000000000000000 00005b77
[32] .debug_str PROGBITS 0000000000000000 00005d2b
[33] .debug_line_str PROGBITS 0000000000000000 00006f79
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$
```

图 32 `-g` 对文件大小和调试段的影响

`-fno-omit-frame-pointer` 在相同的 `-O2` 优化级别下, 分别生成默认版本和加入 `-fno-omit-frame-pointer` 的版本。图 33 中, 默认 `-O2` 版本主要以 `%rsp` 为基准访问局部数据; 加入该选项后出现 `movq %rsp, %rbp`, 后续代码使用 `%rbp` 作为栈帧基址。

```

=== O2 ===
main:
.LFB1813:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    leaq    _ZSt3cin(%rip), %rdi
    subq    $16, %rsp
    .cfi_def_cfa_offset 32
    movq    %fs:40, %rax
    movq    %rax, 8(%rsp)
    xorl    %eax, %eax
    leaq    4(%rsp), %rsi
    call    _ZNSirsERi@PLT
    movl    4(%rsp), %eax
    cmpl    $20, %eax
    ja      .L16
    cmpl    $1, %eax
    jle     .L20
    leal    -2(%rax), %edx
=== O2 + -fno-omit-frame-pointer ===
main:
.LFB1813:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    leaq    _ZSt3cin(%rip), %rdi
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    pushq   %r12
    leaq    -28(%rbp), %rsi
    subq    $24, %rsp
    .cfi_offset 12, -24
    movq    %fs:40, %rax
    movq    %rax, -24(%rbp)
    xorl    %eax, %eax
    call    _ZNSirsERi@PLT
    movl    -28(%rbp), %eax
    cmpl    $20, %eax
    ja      .L16
    cmpl    $1, %eax
    jle     .L20
    leal    -2(%rax), %edx
    movl    $1, %esi
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 33 -fno-omit-frame-pointer 对 main 栈帧的影响

自动并行化与优化报告 使用 -O3 -ftree-parallelize-loops=4 尝试对循环进行自动并行化，同时生成优化报告。图 34 中，GCC 对阶乘循环给出了 couldn't vectorize loop 和 not vectorized，生成的汇编中也没有出现 GOMP/parallel 运行时调用。

factorial 中的 result = mul(result,i) 使用上一轮循环得到的 result，各次迭代之间存在数据依赖，因此这次编译没有将该循环转换为并行执行形式。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$ nl -ba main.cpp | sed -n '17,23p'
echo "==== GCC optimization report ====="
grep "main.cpp:19\\|main.cpp:21" output/parallel_report.txt
echo "==== parallel runtime ====="
grep -E "GOMP|parallel" output/main_parallel.s || echo "no parallel runtime calls"
17
18 //循环
19 for (int i = start; i <= n; i++)
20 //运算、赋值、宏
21 result = mul(result, i);
22 times++;
23 return result;
==== GCC optimization report =====
main.cpp:19:27: missed: couldn't vectorize loop
main.cpp:21:18: missed: not vectorized: relevant stmt not supported: _1 = (long long int) i_15;
main.cpp:19:27: missed: couldn't vectorize loop
main.cpp:21:18: missed: not vectorized: relevant stmt not supported: _13 = (long long int) i_23;
==== parallel runtime =====
no parallel runtime calls
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab$

```

图 34 阶乘循环的优化报告与并行化检查

7. 本小结小结

预处理结果中，N 被替换为 20，`mul(result,i)` 被展开为乘法表达式；定义 `DEBUG` 后，`debug(n)` 生成输出语句。编译阶段得到的 `tree`、`CFG` 和 `RTL` 分别包含程序的树结构、基本块关系以及更接近机器层的表示。目标文件符号表中，函数、零初始化全局变量和外部符号分别出现了 `T`、`B` 和 `U` 标记。

不同编译参数的结果也存在明显差异。`-O2` 版本的 `factorial` 比 `-O0` 版本减少了栈内存读写；加入 `-g` 后，文件中增加了调试相关段；加入 `-fno-omit-frame-pointer` 后，汇编中保留了 `%rbp` 栈帧。静态链接版本约为 2.3 MB，而动态链接版本约为 17 KB，但两种程序输入 5 时都输出 120 和 1。阶乘循环存在前后迭代的数据依赖，优化报告中显示该循环没有被向量化，也没有生成 `GOMP` 并行运行时调用。

(二) 成员 B: C17/Clang/LLVM 编译流程实验

成员 B 使用一份独立的 C17 程序完成本小题。程序里有宏、数组、循环、条件分支和全局变量，适合从源码一路看到预处理、前端分析、IR、汇编、目标文件和链接结果。实验在 Ubuntu 22.04 Docker 容器中完成，使用 Clang/LLVM 14.0.0、GCC 11.4.0 和 Graphviz 2.43.0；中间表示观察 x86-64 LLVM IR，目标代码部分观察 AArch64 汇编和目标文件。

1. 源程序与预处理

程序读入 1 至 8 个整数。宏 `ADJUST(value,bias)` 将数组元素乘以比例常数后加上偏置；`weighted_sum` 根据计算值的奇偶性累加或扣减，并更新全局计数器 `processed_items`。源文件和头文件保存在仓库的 `pipeline/memberB/q1/` 目录中，原始输入为 5 与 1 2 3 4 5。

```

1 #include <stdio.h>
2
3 #include "config.h"
4
5 const int global_bias = 2;
6 int processed_items = 0;
7

```

```
8  int weighted_sum(const int values[], int count)
9  {
10     int total = 0;
11
12     for (int i = 0; i < count; ++i)
13     {
14         int adjusted = ADJUST(values[i], global_bias);
15         if (adjusted % 2 == 0)
16         {
17             total += adjusted;
18         }
19         else
20         {
21             total -= adjusted;
22         }
23         ++processed_items;
24     }
25
26     return total;
27 }
28
29 int main(void)
30 {
31     int count = 0;
32     int values[MAX_ITEMS] = {0};
33
34     if (scanf("%d", &count) != 1 || count < 1 || count > MAX_ITEMS)
35     {
36         return 1;
37     }
38
39     for (int i = 0; i < count; ++i)
40     {
41         if (scanf("%d", &values[i]) != 1)
42         {
43             return 1;
44         }
45     }
46
47     int result = weighted_sum(values, count);
48     printf("result=%d, processed=%d\n", result, processed_items);
49     return 0;
50 }
```

代码 1 实验使用的 C17 源程序

```
1  #ifndef CONFIG_H
```

```
2 #define CONFIG_H
3
4 #define MAX_ITEMS 8
5 #define SCALE_FACTOR 3
6 #define ADJUST(value, bias) ((value) * SCALE_FACTOR + (bias))
7
8 #endif
```

代码 2 宏与数组长度定义

使用预处理选项只执行头文件展开和宏替换, 输出写入 main.i:

```
1 mkdir -p build/memberB/q1
2 clang -E -std=c17 -Ipipeline/memberB/q1/include \
3   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main.i
```

预处理结果中,stdio.h 的声明被展开,MAX_ITEMS 在数组声明中替换为 8,ADJUST(values[i], global_bias) 展开为 ((values[i]) * 3 + (global_bias))。因此 .i 文件保留了原有计算逻辑,但已不再依赖这两个宏定义。

2. 词法、语法与语义分析证据

词法阶段用 Clang 的 token dump 输出 token 种类、词素和源码位置;语法树转储则展示函数、声明、循环、条件分支及表达式的嵌套关系。两个命令都以 -fsyntax-only 避免生成目标文件:

```
1 mkdir -p build/memberB/q1
2 clang -std=c17 -Ipipeline/memberB/q1/include -fsyntax-only \
3   -Xclang -dump-tokens pipeline/memberB/q1/src/main.c \
4   2> build/memberB/q1/tokens.txt
5 clang -std=c17 -Ipipeline/memberB/q1/include -fsyntax-only \
6   -Xclang -ast-dump pipeline/memberB/q1/src/main.c \
7   > build/memberB/q1/ast.txt
```

图 35 中可看到 weighted_sum、for、比较与算术运算符等 token 以及它们在 src/main.c 中的行列位置。图 36 展示了 AST 中的 FunctionDecl、ForStmt、IfStmt、BinaryOperator 等节点;节点的类型注记也表明 Clang 已完成语法解析和类型语义检查。此处观察的是编译器对合法程序的成功分析,不是单独构造错误程序验证诊断。

```
main.i x tokens.txt x
build-linux > tokens.txt > data
1 typedef 'typedef' [StartOfLine] Loc=<usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:1>
1613 semi ';' Loc=<src/main.c:5:26>
1614 int 'int' [StartOfLine] Loc=<src/main.c:6:1>
1615 identifier 'processed_items' [LeadingSpace] Loc=<src/main.c:6:5>
1616 equal '=' [LeadingSpace] Loc=<src/main.c:6:21>
1617 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:6:23>
1618 semi ';' Loc=<src/main.c:6:24>
1619 int 'int' [StartOfLine] Loc=<src/main.c:8:1>
1620 identifier 'weighted_sum' [LeadingSpace] Loc=<src/main.c:8:5>
1621 l_paren '(' Loc=<src/main.c:8:17>
1622 const 'const' Loc=<src/main.c:8:18>
1623 int 'int' [LeadingSpace] Loc=<src/main.c:8:24>
1624 identifier 'values' [LeadingSpace] Loc=<src/main.c:8:28>
1625 l_square '[' Loc=<src/main.c:8:34>
1626 r_square ']' Loc=<src/main.c:8:35>
1627 comma ',' Loc=<src/main.c:8:36>
1628 int 'int' [LeadingSpace] Loc=<src/main.c:8:38>
1629 identifier 'count' [LeadingSpace] Loc=<src/main.c:8:42>
1630 r_paren ')' Loc=<src/main.c:8:47>
1631 l_brace '{' [StartOfLine] Loc=<src/main.c:9:1>
1632 int 'int' [StartOfLine] [LeadingSpace] Loc=<src/main.c:10:5>
1633 identifier 'total' [LeadingSpace] Loc=<src/main.c:10:9>
1634 equal '=' [LeadingSpace] Loc=<src/main.c:10:15>
1635 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:10:17>
1636 semi ';' Loc=<src/main.c:10:18>
1637 for 'for' [StartOfLine] [LeadingSpace] Loc=<src/main.c:12:5>
1638 l_paren '(' [LeadingSpace] Loc=<src/main.c:12:9>
1639 int 'int' Loc=<src/main.c:12:10>
1640 identifier 'i' [LeadingSpace] Loc=<src/main.c:12:14>
1641 equal '=' [LeadingSpace] Loc=<src/main.c:12:16>
1642 numeric_constant '0' [LeadingSpace] Loc=<src/main.c:12:18>
1643 semi ';' Loc=<src/main.c:12:19>
1644 identifier 'i' [LeadingSpace] Loc=<src/main.c:12:21>
1645 less '<' [LeadingSpace] Loc=<src/main.c:12:23>
1646 identifier 'count' [LeadingSpace] Loc=<src/main.c:12:25>
1647 semi ';' Loc=<src/main.c:12:30>
1648 plusplus '++' [LeadingSpace] Loc=<src/main.c:12:32>
```

图 35 Clang token dump 及源码位置

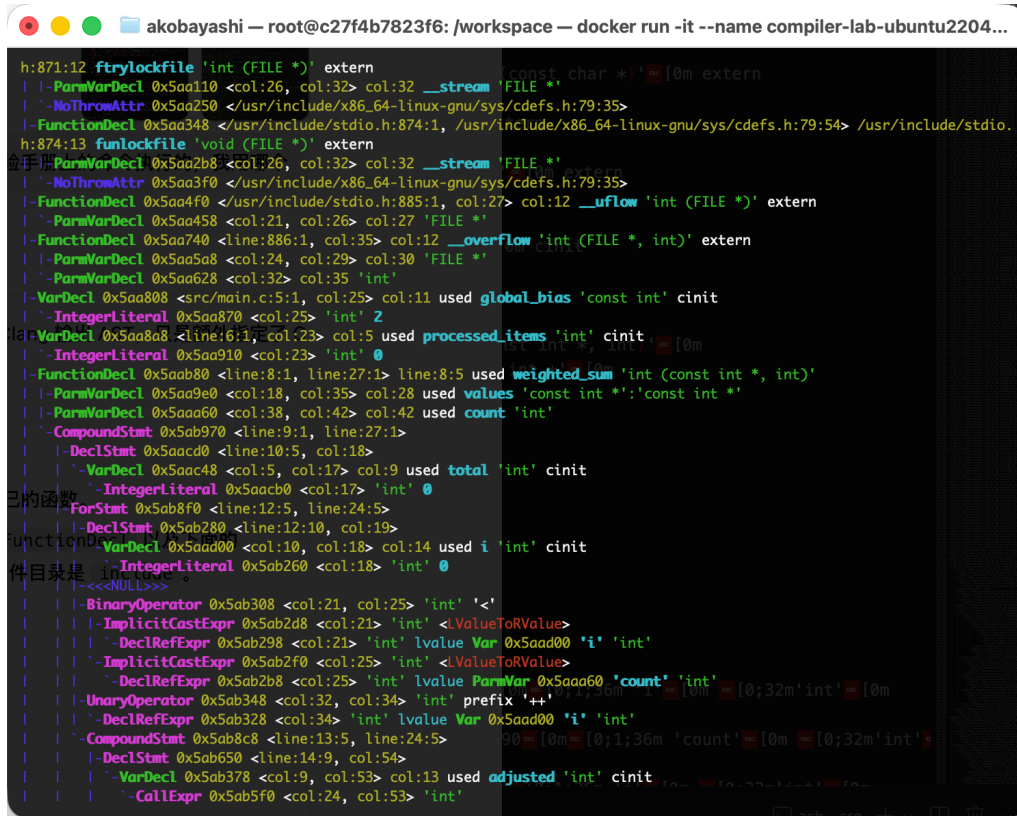


图 36 Clang AST 中的函数、循环与条件节点

3. 控制流图与 LLVM IR

为直接观察控制流图，GCC 在编译时写出 tree/CFG 图文件，再由 Graphviz 将 .dot 文件绘制为图 37。其中循环回边对应 for 的重复执行，条件分支对应奇偶判断以及 main 中的输入合法性检查。

```
1 mkdir -p build/memberB/q1/gcc-graphs
2 gcc -O0 -Ipipeline/memberB/q1/include -fdump-tree-all-graph \
3   -dumpdir build/memberB/q1/gcc-graphs/ -dumppbase main-gcc \
4   -c pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-gcc.o
5 dot -Tpng build/memberB/q1/gcc-graphs/main-gcc.015t.cfg.dot \
6   -o build/memberB/q1/gcc-graphs/main-cfg.png
```

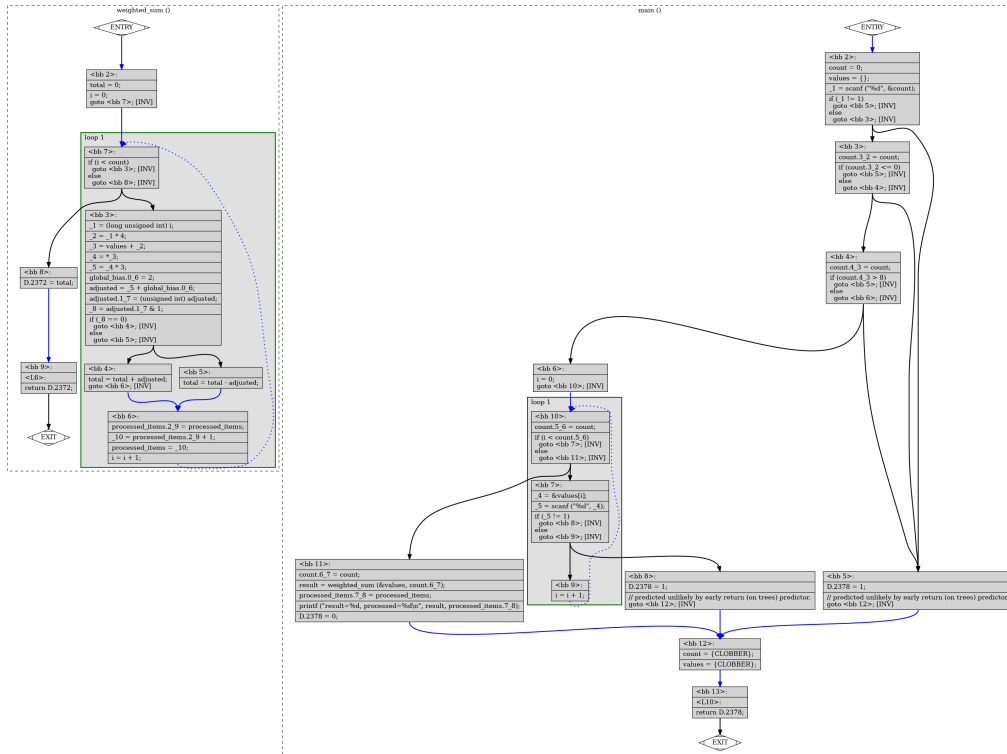


图 37 GCC 生成并由 Graphviz 绘制的控制流图

LLVM IR 由 Clang 前端从同一份 C 源码生成。-O0 保留较多局部存储、load/store 与显式控制流，便于将 IR 与源码逐句对应；图 38 展示了函数定义、数组操作、比较和分支。

```
1 clang -std=c17 -Ipipeline/memberB/q1/include -O0 -S -emit-llvm \
2 pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-O0.ll
```

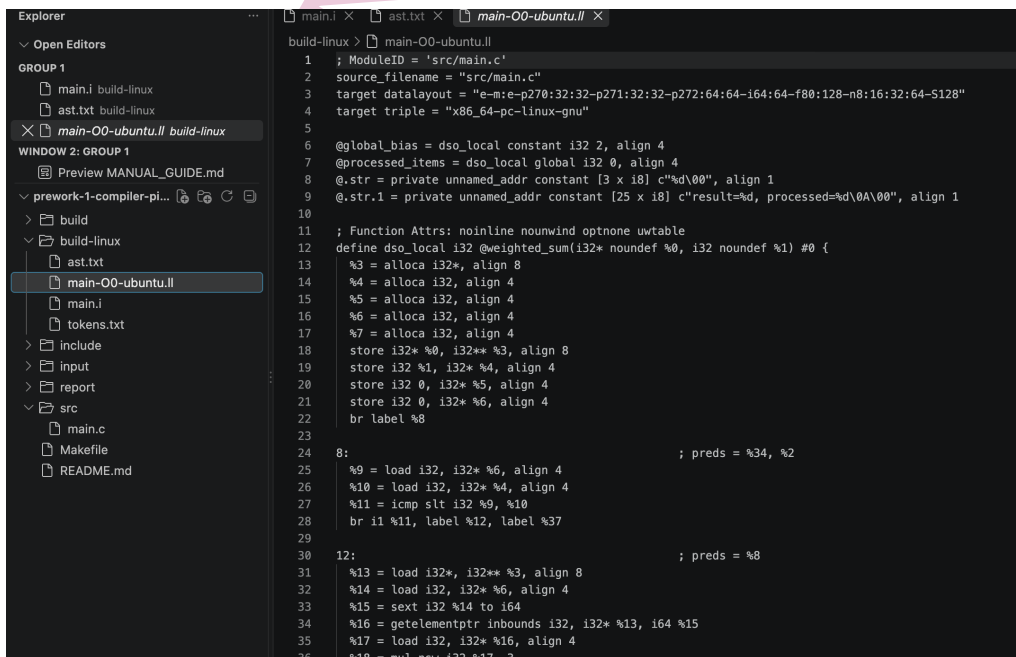


图 38 未优化 LLVM IR: 函数、基本块与内存操作

4. 优化前后比较与目标代码生成

以相同源程序分别使用 `-O0` 和 `-O2` 生成 IR, 构成参数对照。图 39 中, 优化版本的局部计算更倾向于使用 SSA 临时值, 减少部分冗余栈读写, 并带有 `nsw` 等运算属性; 函数属性、循环组织也有变化。优化并不改变本程序的可观察输入输出, 但 IR 不再是源代码逐行翻译的形式。

```
1 clang -std=c17 -Ipipeline/memberB/q1/include -O2 -S -emit-llvm \
2   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-O2.ll
```

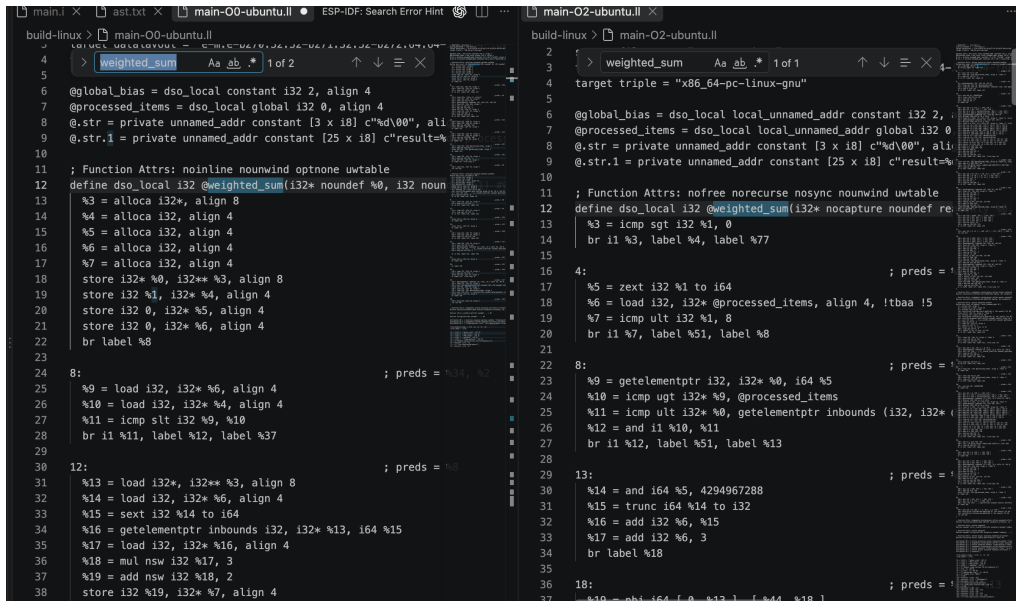


图 39 同一函数在 `-O0` 与 `-O2` 下的 LLVM IR 对照

目标代码阶段使用 LLVM 后端将 IR 映射为 AArch64 汇编。该汇编出现了 32 位/64 位通用寄存器、条件分支和数组寻址指令; 汇编器再把文本指令编码为 AArch64 机器码。目标文件为可重定位 ELF, 仍包含需要链接器处理的符号与重定位项。

```
1 clang --target=aarch64-linux-gnu -std=c17 \
2   -Ipipeline/memberB/q1/include -O2 -S -emit-llvm \
3   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-O2-aarch64.ll
4 llc -O2 build/memberB/q1/main-O2-aarch64.ll \
5   -o build/memberB/q1/main-O2-aarch64.s
6 aarch64-linux-gnu-gcc -c build/memberB/q1/main-O2-aarch64.s \
7   -o build/memberB/q1/main-O2-aarch64.o
8 aarch64-linux-gnu-readelf -h build/memberB/q1/main-O2-aarch64.o
9 aarch64-linux-gnu-readelf -Ws -rW build/memberB/q1/main-O2-aarch64.o
```

图 40 是 AArch64 汇编片段; 图 41 的 ELF 头中 `Type: REL` 与 `Machine: AArch64` 说明产物是尚未链接的可重定位目标文件。图 42 中, 自定义函数和全局变量进入符号表, `printf/scanf` 等外部函数仍是未定义符号; `R_AARCH64_*` 重定位项记录了链接时需要修补的位置。

```

11 // %bb.1:
12 b.ls .LBB0_4
13
14 // %bb.2:
15 adrp x11, processed_items
16 add x8, x0, x10, lsl #2
17 add x11, x11, :lo12:processed_items
18 cmp x8, x11
19 b.ls .LBB0_9
20
21 // %bb.3:
22 adrp x8, processed_items+4
23 add x8, x8, :lo12:processed_items+4
24 cmp x0, x8
25 b.hs .LBB0_9
26
27 .LBB0_4:
28 mov x11, xzr
29 mov w8, wzr
30 mov w13, w12
31
32 .LBB0_5: // %.preheader
33 add w12, w13, #1
34 sub x10, x10, x11
35 add x11, x0, x11, lsl #2
36 mov w13, #-2
37
38 .LBB0_6: // =>This Inner Loop Header: Depth=1
39 ldr w14, [x11], #4
40 str w12, [x9, :lo12:processed_items]
41 add w12, w12, #1
42 add w14, w14, w14, lsl #1
43 add w15, w14, #2
44 sub w16, w13, w14
45 tst w14, #0x1
46 csel w14, w15, w16, eq
47 subs x10, x10, #1
48 add w8, w14, w8
49 b.ne .LBB0_6
50
51 .LBB0_7:
52 mov w0, w8

```

图 40 LLVM 后端生成的 AArch64 汇编

```

root@c27f4b7823f6:/workspace/build-linux# readelf -h main-O2-aarch64.o
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:       ELF64
  Data:       2's complement, little endian
  Version:    1 (current)
  OS/ABI:     UNIX System V
  ABI Version: 0
  Type:       REL (Relocatable file)
  Machine:    AArch64
  Version:    0x1
  Entry point address: 0x0
  Start of program headers: 0 (bytes into file)
  Start of section headers: 2192 (bytes into file)
  Flags:      0x0
  Size of this header: 64 (bytes)
  Size of program headers: 0 (bytes)
  Number of program headers: 0
  Size of section headers: 64 (bytes)
  Number of section headers: 14
  Section header string table index: 13
root@c27f4b7823f6:/workspace/build-linux#

```

图 41 AArch64 可重定位目标文件的 ELF 头

```

root@c27f4b7823f6:/workspace/build-linux# aarch64-linux-gnu-readelf -Ws main-O2-aarch64.o
[aarch64-linux-gnu-readelf -rW main-O2-aarch64.o]

Symbol table '.symtab' contains 20 entries:
  Num:      Value              Size Type      Bind   Vis      Ndx Name
   0: 0000000000000000      0 NOTYPE   LOCAL  DEFAULT UND
   1: 0000000000000000      0 FILE    LOCAL  DEFAULT ABS main.c
   2: 0000000000000000      0 SECTION LOCAL  DEFAULT 1 .text
   3: 0000000000000000      0 SECTION LOCAL  DEFAULT 3 .data
   4: 0000000000000000      0 SECTION LOCAL  DEFAULT 4 .bss
   5: 0000000000000000      0 NOTYPE   LOCAL  DEFAULT 1 $x
   6: 0000000000000000      0 SECTION LOCAL  DEFAULT 5 .rodata
   7: 0000000000000000      0 NOTYPE   LOCAL  DEFAULT 5 $d
   8: 0000000000000000      0 NOTYPE   LOCAL  DEFAULT 4 $d
   9: 0000000000000000      0 SECTION LOCAL  DEFAULT 6 .rodata.str1.1
  10: 0000000000000000      0 SECTION LOCAL  DEFAULT 8 .note.GNU-stack
  11: 0000000000000014      0 NOTYPE   LOCAL  DEFAULT 9 $d
  12: 0000000000000000      0 SECTION LOCAL  DEFAULT 9 .eh_frame
  13: 0000000000000000      0 SECTION LOCAL  DEFAULT 7 .comment
  14: 0000000000000000     296 FUNC    GLOBAL DEFAULT 1 weighted_sum
  15: 0000000000000000      4 OBJECT  GLOBAL DEFAULT 4 processed_items
  16: 0000000000000128     432 FUNC    GLOBAL DEFAULT 1 main
  17: 0000000000000000      0 NOTYPE   GLOBAL DEFAULT UND __isoc99_scanf
  18: 0000000000000000      0 NOTYPE   GLOBAL DEFAULT UND printf
  19: 0000000000000000      4 OBJECT  GLOBAL DEFAULT 5 global_bias

Relocation section '.rela.text' at offset 0x5e0 contains 22 entries:
  Offset          Info              Type              Symbol's Value  Symbol's Name + Addend
 0000000000000008 0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
 0000000000000014 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 000000000000001c 0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
 0000000000000024 0000000f00000115 R_AARCH64_ADD_ABS_LO12_NC 0000000000000000 processed_items + 0
 0000000000000030 0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 4
 0000000000000034 0000000f00000115 R_AARCH64_ADD_ABS_LO12_NC 0000000000000000 processed_items + 4
 0000000000000060 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 0000000000000114 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 0000000000000140 0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 0
 0000000000000144 0000000900000115 R_AARCH64_ADD_ABS_LO12_NC 0000000000000000 .rodata.str1.1 + 0
 0000000000000154 000000110000011b R_AARCH64_CALL26 0000000000000000 __isoc99_scanf + 0
 000000000000017c 0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 0
 0000000000000188 0000000900000115 R_AARCH64_ADD_ABS_LO12_NC 0000000000000000 .rodata.str1.1 + 0
 0000000000000194 000000110000011b R_AARCH64_CALL26 0000000000000000 __isoc99_scanf + 0
 00000000000001c0 0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
 00000000000001c8 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 00000000000001e4 0000000f00000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 processed_items + 0
 00000000000001ec 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 00000000000002b0 0000000f0000011d R_AARCH64_LDST32_ABS_LO12_NC 0000000000000000 processed_items + 0
 00000000000002b4 0000000900000113 R_AARCH64_ADR_PREL_PG_HI21 0000000000000000 .rodata.str1.1 + 3
 00000000000002b8 0000000900000115 R_AARCH64_ADD_ABS_LO12_NC 0000000000000000 .rodata.str1.1 + 3
 00000000000002bc 000000120000011b R_AARCH64_CALL26 0000000000000000 printf + 0

Relocation section '.rela.eh_frame' at offset 0x7f0 contains 2 entries:
  Offset          Info              Type              Symbol's Value  Symbol's Name + Addend
 000000000000001c 0000000200000105 R_AARCH64_PREL32 0000000000000000 .text + 0
 0000000000000030 0000000200000105 R_AARCH64_PREL32 0000000000000000 .text + 128
root@c27f4b7823f6:/workspace/build-linux#

```

图 42 目标文件的符号表与重定位表

5. 链接、反汇编与运行验证

将目标文件与 C 运行时库链接后，外部函数调用可解析为可执行文件中的动态链接入口。图 43 对照了链接前后的反汇编：目标文件中保留针对 `printf` 的调用重定位，链接后的程序则显示具体地址和 `printf@plt` 调用入口。AArch64 可执行文件不能直接作为 macOS arm64 原生程序运行；本实验在 Ubuntu 容器中通过 QEMU 用户态执行，并用同一组输入检查结果。

```

1 clang -std=c17 -Ipipeline/memberB/q1/include -O2 \
2   pipeline/memberB/q1/src/main.c -o build/memberB/q1/main-native
3 build/memberB/q1/main-native < pipeline/memberB/q1/input/sample.txt
4 aarch64-linux-gnu-gcc -no-pie build/memberB/q1/main-O2-aarch64.o \
5   -o build/memberB/q1/main-aarch64
6 qemu-aarch64 -L /usr/aarch64-linux-gnu \

```

```
7 | build/memberB/q1/main-aarch64 < pipeline/memberB/q1/input/sample.txt
```

样例数据为 1, 2, 3, 4, 5, 调整值分别为 5, 8, 11, 14, 17, 按奇偶规则计算得到 -11, 处理数为 5。报告截图中的目标文件/链接证据来自 AArch64 目标; 以上原生编译命令用于验证同一 C 源程序的运行语义。

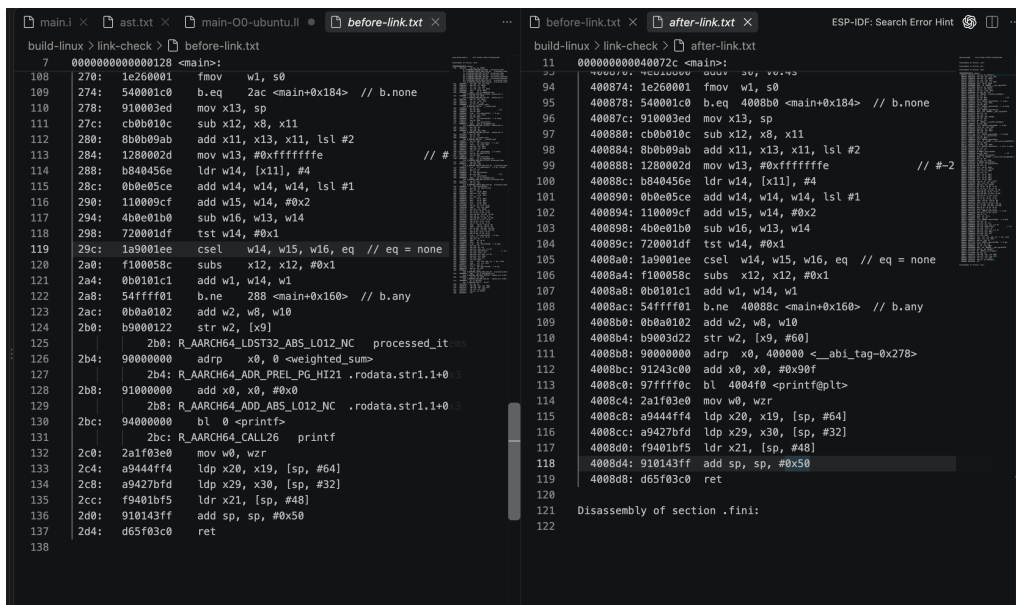


图 43 链接前目标文件与链接后程序的反汇编对照

6. 阶段观察小结

本小题中, MAX_ITEMS 在预处理文件中替换为 8, ADJUST(value, bias) 展开为“元素乘以 SCALE_FACTOR 后加 global_bias”的表达式。weighted_sum 的 token、AST、CFG 和 LLVM IR 分别保留了循环、奇偶分支和对 processed_items 的更新; -O2 相比 -O0 减少了重复的内存读写。AArch64 目标文件中的符号和重定位项、以及链接后的反汇编说明了 printf 等外部调用在链接阶段得到处理。样例输入 1 2 3 4 5 的结果为 result=-11, processed=5, 与源码中的计算一致。

四、 小题 2: LLVM IR 与 RISC-V 汇编编程

小题 2 中, 我们共同编写了一个 SysY 程序, 程序读取整数数组, 计算其中所有偶数之和, 并统计正数个数。成员 A 根据该程序编写 LLVM IR, 成员 B 编写 RISC-V 汇编, 两种实现均调用 SysY 运行时库中的输入输出函数, 并使用相同的测试数据检查运行结果。

(一) SysY 示例程序与测试数据

我们编写的 SysY 程序包含数组、赋值、取模运算、关系判断、if 条件分支、while 循环、函数、数组参数和函数调用。程序通过 getint 读取输入, 使用 putint 和 putch 输出结果。数组长度为 10, 测试时取 $0 \leq n \leq 10$ 。

```
1 | // 求偶数和
2 | int sum_even(int a[], int n)
```

```
3 {
4     int i;
5     int sum;
6     i=0;
7     sum=0;
8     while (i<n)
9     {
10         if (a[i]%2==0)
11             sum=sum+a[i];
12         i=i+1;
13     }
14     return sum;
15 }
16
17 // 统计正数个数
18 int count_pos(int a[],int n)
19 {
20     int i;
21     int cnt;
22     i=0;
23     cnt=0;
24     while (i<n)
25     {
26         if (a[i]>0)
27             cnt=cnt+1;
28         i=i+1;
29     }
30     return cnt;
31 }
32
33 int main()
34 {
35     int a[10];
36     int n;
37     int i;
38     int s;
39     int c;
40
41     // 输入
42     n=getint();
43     i=0;
44     while (i<n)
45     {
46         a[i]=getint();
47         i=i+1;
48     }
```

```

49
50 // 计算
51 s=sum_even(a,n);
52 c=count_pos(a,n);
53
54 // 输出
55 putint(s);
56 putchar(10);
57 putint(c);
58 putchar(10);
59
60 return 0;
61 }

```

代码 3 SysY 示例程序 main.sy

我们使用三组数据测试 LLVM IR 和 RISC-V 汇编程序。第 1 组包含正数、负数、奇数和偶数；第 2 组包含负偶数，用于检查负偶数是否正常参与求和；第 3 组全部为正奇数，偶数和应为 0。三组输入及预期结果如表所示。

表 2 小题 2 测试数据

组别	输入（第一项为 n ）	偶数和	正数个数
1	5; 1 2 -3 4 6	12	4
2	4; -2 3 8 -5	6	2
3	5; 1 3 5 7 9	0	5

（二） LLVM IR 实现

本部分由成员 A 负责。根据前面的 SysY 程序编写 `llvm/memberB/main.ll`，保留 `sum_even`、`count_pos` 和 `main` 三个函数。循环和条件分支使用基本块实现，循环变量和计算结果使用 SSA 临时值和 `phi` 指令传递，输入输出调用 SysY 运行时库中的 `getint`、`putint` 和 `putch`。

1. SysY 与 LLVM IR 的对应

表中列出了 SysY 程序中的主要结构及其在 LLVM IR 中的实现方式。

表 3 SysY 结构与 LLVM IR 的对应关系

SysY 结构	LLVM IR 指令/结构	实现方式
int a[10]	alloca [10 x i32]	在 main 中分配 10 个 32 位整数的数组空间
a[i]	getelementptr, load/store	计算数组元素地址并进行读写
while (i<n)	br, icmp slt	判断循环条件并在基本块之间跳转
a[i] % 2 == 0	srem, icmp eq	计算余数并判断当前元素是否为偶数
if 条件分支	条件 br 与 phi	根据判断结果进入不同基本块，并合并后续计算结果
函数参数/返回值	i32*, i32, ret	传递数组首地址和长度，并返回计算结果
输入/输出	call	调用 getint、putint 和 putch

在 sum_even 中，循环头使用 phi 保存循环下标和当前累计和。程序先通过 srem 计算数组元素对 2 的余数，再根据判断结果决定是否更新累计和。count_pos 使用相同的循环结构，通过比较元素是否大于 0 更新正数个数。

main 使用 alloca 分配数组空间，通过 getint 读入 n 和数组元素，再将数组首地址和 n 传给 sum_even 和 count_pos。两个函数返回结果后，使用 putint 和 putch 输出。

2. LLVM IR 程序

成员 A 编写的完整 LLVM IR 如下：

```

1 ; Hand-written LLVM IR equivalent to report/sections/q2_common_sysy.tex.
2 ; The SysY runtime provides getint, putint and putch.
3
4 target triple = "x86_64-pc-linux-gnu"
5
6 declare i32 @getint()
7 declare void @putint(i32)
8 declare void @putch(i32)
9
10 define i32 @sum_even(i32* %a, i32 %n) {
11 entry:
12   br label %loop.cond
13
14 loop.cond:
15   %i = phi i32 [ 0, %entry ], [ %i.next, %if.end ]
16   %sum = phi i32 [ 0, %entry ], [ %sum.next, %if.end ]
17   %more = icmp slt i32 %i, %n
18   br i1 %more, label %loop.body, label %loop.exit
19
20 loop.body:
21   %element.ptr = getelementptr inbounds i32, i32* %a, i32 %i
22   %value = load i32, i32* %element.ptr, align 4
23   %remainder = srem i32 %value, 2

```

```
24  %is.even = icmp eq i32 %remainder, 0
25  br i1 %is.even, label %if.then, label %if.else
26
27  if.then:
28  %sum.add = add nsw i32 %sum, %value
29  br label %if.end
30
31  if.else:
32  br label %if.end
33
34  if.end:
35  %sum.next = phi i32 [ %sum.add, %if.then ], [ %sum, %if.else ]
36  %i.next = add nsw i32 %i, 1
37  br label %loop.cond
38
39  loop.exit:
40  ret i32 %sum
41 }
42
43 define i32 @count_pos(i32* %a, i32 %n) {
44 entry:
45  br label %loop.cond
46
47  loop.cond:
48  %i = phi i32 [ 0, %entry ], [ %i.next, %if.end ]
49  %count = phi i32 [ 0, %entry ], [ %count.next, %if.end ]
50  %more = icmp slt i32 %i, %n
51  br i1 %more, label %loop.body, label %loop.exit
52
53  loop.body:
54  %element.ptr = getelementptr inbounds i32, i32* %a, i32 %i
55  %value = load i32, i32* %element.ptr, align 4
56  %is.positive = icmp sgt i32 %value, 0
57  br i1 %is.positive, label %if.then, label %if.else
58
59  if.then:
60  %count.inc = add nsw i32 %count, 1
61  br label %if.end
62
63  if.else:
64  br label %if.end
65
66  if.end:
67  %count.next = phi i32 [ %count.inc, %if.then ], [ %count, %if.else ]
68  %i.next = add nsw i32 %i, 1
69  br label %loop.cond
```

```
70
71 loop.exit:
72     ret i32 %count
73 }
74
75 define i32 @main() {
76     entry:
77         %a = alloca [10 x i32], align 16
78         %n = call i32 @getint()
79         br label %input.cond
80
81     input.cond:
82         %i = phi i32 [ 0, %entry ], [ %i.next, %input.body ]
83         %more = icmp slt i32 %i, %n
84         br i1 %more, label %input.body, label %compute
85
86     input.body:
87         %element.ptr = getelementptr inbounds [10 x i32], [10 x i32]* %a, i32 0,
            i32 %i
88         %value = call i32 @getint()
89         store i32 %value, i32* %element.ptr, align 4
90         %i.next = add nsw i32 %i, 1
91         br label %input.cond
92
93     compute:
94         %first = getelementptr inbounds [10 x i32], [10 x i32]* %a, i32 0, i32 0
95         %sum = call i32 @sum_even(i32* %first, i32 %n)
96         %positive = call i32 @count_pos(i32* %first, i32 %n)
97         call void @putint(i32 %sum)
98         call void @putch(i32 10)
99         call void @putint(i32 %positive)
100        call void @putch(i32 10)
101        ret i32 0
102 }
```

代码 4 LLVM IR 程序 main.ll

3. 编译、链接与运行

实验环境为 Ubuntu 22.04, LLVM/Clang 版本为 14, LLVM IR 的 target triple 为 x86_64-pc-linux-gnu。SysY 运行时使用仓库中的 riscv/lib/sylib.c, 相关编译和测试命令写在 llvm/memberB/Makefile 中, 生成文件保存在 build/ 目录。

```
1 cd llvm/memberB
2 make all
3 llvm-readelf -h build/main.o
4 llvm-readelf -rW build/main.o
```

```

5 nm build/main-native | grep -E "getint|putint|putch|sum_even|count_pos| main$
   "
6 make test

```

llvm-as 将 main.ll 转换为 bitcode, Clang 再生成本机目标文件并与 SysY 运行时链接为可执行文件。运行时库同时编译为共享库, 供 lli 执行 LLVM IR 时加载。

使用 llvm-readelf 检查目标文件的 ELF 信息和重定位项, 使用 nm 检查 sum_even、count_pos、main 以及 getint、putint、putch 等符号。

4. 测试结果

使用 make test-native 和 make test-lli 分别运行本机可执行文件和 LLVM lli。两种执行方式均使用前面确定的三组测试数据。

表 4 LLVM IR 两种执行方式的测试结果

组别	输入	预期输出	native / lli 实际输出
1	5; 1 2 -3 4 6	12; 4	12; 4
2	4; -2 3 8 -5	6; 2	6; 2
3	5; 1 3 5 7 9	0; 5	0; 5

第 1 组输入中同时包含正数、负数、奇数和偶数, 程序输出偶数和 12、正数个数 4。第 2 组中包含负偶数 -2, 计算结果为偶数和 6、正数个数 2。第 3 组全部为正奇数, 因此偶数和为 0, 正数个数为 5。

三组数据在 native 和 lli 下的输出分别为 12/4、6/2 和 0/5, 与前面的测试结果一致。

5. 本部分小结

LLVM IR 中, sum_even 使用 srem 判断偶数, 并通过 phi 保存循环下标和累计结果; count_pos 使用比较和条件分支统计正数个数; main 使用 getelementptr 计算数组元素地址, 并调用 SysY 运行时函数完成输入输出。

三组测试数据在 native 和 lli 两种方式下均得到 12/4、6/2 和 0/5, 数组访问、循环、条件分支和函数调用的结果与前面的 SysY 程序一致。

(三) RISC-V 汇编程序

本部分由成员 B 负责。实验中编写了与前面 SysY 程序对应的 RISC-V 汇编代码, 完成数组输入、偶数求和和正数个数统计, 并与 SysY 运行时库链接。程序使用 qemu-riscv64 运行, 先使用三组基础数据进行测试, 再补充 100 组数据, 并检查目标文件和最终可执行文件中的函数符号。

1. 寄存器与主要指令

数组首地址和长度分别放在 a0 和 a1 中, 函数返回值放在 a0 中。main 中使用 s0 保存 n, s1 保存循环变量, s2 保存数组基址, 并在栈中保存 ra 和 s0--s2。栈帧大小为 80 B, 保持 16 B 对齐。

表 5 SysY 结构与 RISC-V 汇编写法

SysY 结构	RISC-V 指令/寄存器	作用
函数参数 a、n	a0、a1	传递数组首地址和数组长度
return	a0	保存函数返回值
a[i]	slli + add + lw/sw	计算数组元素地址并进行读写
while	条件分支 + 标签 + 回跳	控制循环执行
a[i]%2==0	remw + bnez	计算余数并判断是否为偶数
a[i]>0	blez	判断元素是否为正数
getint/putint/ putch	call	调用 SysY 运行时库函数
main 栈帧	addi sp,sp,-80 / +80	保存 ra 和 s0--s2

访问 $a[i]$ 时, 先使用 `slli` 将下标左移 2 位得到 $i \times 4$, 再使用 `add` 与数组基址相加, 最后通过 `lw` 或 `sw` 读写数组元素。`sum_even` 使用 `remw` 计算元素除以 2 的余数, 通过 `bnez` 跳过奇数; `count_pos` 使用 `blez` 跳过非正数。

2. RISC-V 汇编代码

成员 B 编写的 RISC-V 汇编代码如下:

```

1 .option nopic
2 .attribute stack_align,16
3 .text
4
5 .globl sum_even
6 .type sum_even,@function
7 sum_even:
8 #初始化
9     li t0,0
10    li t1,0
11 .Lse_loop:
12    bge t0,a1,.Lse_end
13    slli t2,t0,2
14    add t3,a0,t2
15    lw t4,0(t3)
16 #判断偶数
17    li t5,2
18    remw t6,t4,t5
19    bnez t6,.Lse_next
20    addw t1,t1,t4
21 .Lse_next:
22    addi t0,t0,1
23    j .Lse_loop
24 .Lse_end:
25    mv a0,t1
26    ret
27 .size sum_even,.-sum_even

```

```
28
29 .globl count_pos
30 .type count_pos,@function
31 count_pos:
32 #初始化
33     li t0,0
34     li t1,0
35 .Lcp_loop:
36     bge t0,a1,.Lcp_end
37     slli t2,t0,2
38     add t3,a0,t2
39     lw t4,0(t3)
40 #统计正数
41     blez t4,.Lcp_next
42     addiw t1,t1,1
43 .Lcp_next:
44     addi t0,t0,1
45     j .Lcp_loop
46 .Lcp_end:
47     mv a0,t1
48     ret
49 .size count_pos,.-count_pos
50
51 .globl main
52 .type main,@function
53 main:
54 #开辟栈空间
55     addi sp,sp,-80
56     sd ra,72(sp)
57     sd s0,64(sp)
58     sd s1,56(sp)
59     sd s2,48(sp)
60
61     mv s2,sp
62
63 #输入n
64     call getint
65     mv s0,a0
66     li s1,0
67
68 #输入数组
69 .Linput:
70     bge s1,s0,.Linput_end
71     call getint
72     slli t0,s1,2
73     add t0,s2,t0
```

```
74     sw a0,0(t0)
75     addi s1,s1,1
76     j .Linput
77
78 .Linput_end:
79 #求偶数和
80     mv a0,s2
81     mv a1,s0
82     call sum_even
83     call putint
84     li a0,10
85     call putch
86
87 #统计正数
88     mv a0,s2
89     mv a1,s0
90     call count_pos
91     call putint
92     li a0,10
93     call putch
94
95     li a0,0
96     ld s2,48(sp)
97     ld s1,56(sp)
98     ld s0,64(sp)
99     ld ra,72(sp)
100    addi sp,sp,80
101    ret
102 .size main,.-main
```

代码 5 RISC-V 汇编程序 main.s

3. 汇编与链接

使用 RISC-V 交叉编译器将 main.s 汇编为目标文件 main.o:

```
1 riscv64-unknown-elf-gcc main.s -c -o main.o -w
2 file main.o
```

file 的结果中包含 ELF 64-bit、relocatable 和 UCB RISC-V, 生成的 main.o 为 64 位 RISC-V 可重定位目标文件。

程序调用了 SysY 运行时库中的 getint、putint 和 putch。使用下面的命令将 main.o 与 libsysy_riscv.a 静态链接:

```
1 riscv64-unknown-elf-gcc main.o -o main \
2   -L./lib -lsysy_riscv \
3   -static -mcmodel=medany \
4   -Wl,--no-relax,-Ttext=0x90000000
```

第一次链接时,终端使用的是 SiFive GCC-Metal 10.2.0 工具链,链接器报告 zmmul ISA 属性不兼容。使用 `type -a` 检查工具链路径后,将 `/home/lenovo/riscv-elf-toolchains/bin` 放到 PATH 前端,并重新生成 `main.o`,之后程序正常链接。

目标文件检查和链接结果如图 44 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ cd /mnt/d/compilerLab/riscv
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ rm -f main.o main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-gcc main.s -c -o main.o -w
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ file main.o
main.o: ELF 64-bit LSB relocatable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-gcc main.o -o main \
-L./lib -lsys_riscv \
-static -mcmodel=medany \
-Wl,--no-relax,-Ttext=0x90000000
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 44 RV64 目标文件检查与 SysY 运行时库链接

4. 运行结果

使用 `qemu-riscv64` 运行链接后的程序:

```
1 qemu-riscv64 ./main
```

三组测试数据的运行结果如图 45 至图 47 所示。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ file main
ls -lh main
main: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked, with debug_info, not stripped
-rwxrwxrwx 1 lenovo lenovo 164K Sep 25 02:33 main
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
5
1
2
-3
4
6
12
4
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 45 可执行文件类型与第 1 组运行结果

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
4
-2
3
8
-5
6
2
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 46 第 2 组运行结果

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ qemu-riscv64 ./main
5
1
3
5
7
9
0
5
TOTAL: 0H-0M-0S-0us
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 47 第 3 组运行结果

表 6 RISC-V 程序测试结果

组别	输入	预期输出	实际输出
1	5; 1 2 -3 4 6	12; 4	12; 4
2	4; -2 3 8 -5	6; 2	6; 2
3	5; 1 3 5 7 9	0; 5	0; 5

第 1 组同时包含正数、负数、奇数和偶数，程序输出 12/4。第 2 组包含负偶数 -2，程序输出 6/2。第 3 组全部为正奇数，程序输出 0/5。三组结果与前面的 SysY 程序和 LLVM IR 程序一致。

除三组基础数据外，又使用 100 组数据测试 RISC-V 程序。其中 20 组包括空数组、零、正负数和最大长度等情况，另外 80 组使用不同的连续整数和正负数组合。

表 7 RISC-V 扩展测试结果

用例类别	用例数	通过数	失败数
特殊数据	20	20	0
普通数据	80	80	0
合计	100	100	0

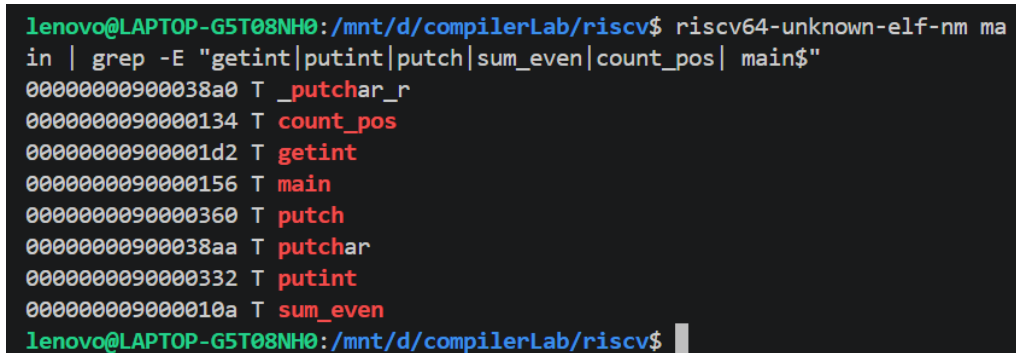
100 组测试全部通过,完整的输入、预期输出和实际输出保存在 riscv/test_results_100.md 中。

5. 符号检查

使用 nm 查看最终可执行文件中的函数符号：

```
1 riscv64-unknown-elf-nm main \
2 | grep -E "getint|putint|putch|sum_even|count_pos| main$"
```

图 48 中可以看到 sum_even、count_pos 和 main，同时也包含 getint、putint 和 putch。



```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$ riscv64-unknown-elf-nm main | grep -E "getint|putint|putch|sum_even|count_pos| main$"
00000000900038a0 T _putchar_r
0000000090000134 T count_pos
00000000900001d2 T getint
0000000090000156 T main
0000000090000360 T putch
00000000900038aa T putchar
0000000090000332 T putint
000000009000010a T sum_even
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/riscv$
```

图 48 最终可执行文件中的程序函数与 SysY 运行时库符号

6. 本部分小结

RISC-V 汇编中使用 slli、add 和 lw 访问数组元素，使用 remw 和 bnez 判断偶数，使用 blez 判断正数。main 使用 80 B 栈帧保存 ra 和 s0--s2，函数调用时通过 a0 和 a1 传递数组地址和长度。

main.s 汇编为 main.o 后，与 libsysy_riscv.a 静态链接生成 RISC-V 可执行文件。三组基础测试分别输出 12/4、6/2 和 0/5，100 组扩展测试全部通过。最终可执行文件的符号表中可以同时看到程序中的函数和 SysY 运行时库中的输入输出函数。

五、进阶要求：观察 AI 编译器 MLIR 中端处理过程

本节完成实验的进阶要求。我们使用 MLIR 14，以一个基础算术程序和固定尺寸的矩阵乘法程序为例，观察中端表示由高层 linalg.matmul 逐步 Lowering 到循环、控制流、LLVM Dialect、标准 LLVM IR 和 x86-64 目标文件的过程。实验在 WSL2 Ubuntu 22.04 中完成，工具为 mlir-opt-14、mlir-translate-14 和 Clang 14。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ pwd
clang --version | head -n 1
mlir-opt-14 --version
mlir-translate-14 --version
/mnt/d/compilerLab/mlir
Ubuntu clang version 14.0.0-1ubuntu1.1
Ubuntu LLVM version 14.0.0

Optimized build.
Default target: x86_64-pc-linux-gnu
Host CPU: goldmont
Ubuntu LLVM version 14.0.0

Optimized build.
Default target: x86_64-pc-linux-gnu
Host CPU: goldmont
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$

```

图 49 MLIR 14 实验环境与工具版本

（一）任务与分工

进阶部分使用同一组 MLIR 材料。成员 A 整理基础 MLIR 程序和高层矩阵乘法表示，执行 `canonicalize` 与 `convert-linalg-to-loops`，记录高层操作展开为三层循环的结果。成员 B 继续检查 `convert-scf-to-std` 后的基本块和分支，完成 LLVM Dialect、标准 LLVM IR 与目标文件的核对。两人共同对照每个阶段保存的文件和终端输出，将实验过程写入报告。

（二）基础 MLIR 与 Canonicalization

`mlir/test.mlir` 中定义了两个 `i32` 常量和一次 `arith.addi`。加法结果没有被后续操作使用，适合观察规范化前后的变化。

```

1 module {
2   func @main() {
3     %0 = arith.constant 1 : i32
4     %1 = arith.constant 2 : i32
5     %2 = arith.addi %0, %1 : i32
6     return
7   }
8 }

```

代码 6 基础 MLIR 程序 test.mlir

先直接查看程序：

```

1 mlir-opt-14 test.mlir

```

输出中保留 `module`、`func`、`arith.constant` 和 `arith.addi`，说明程序能够被 MLIR 工具读取。图50给出了该命令的输出。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 test.mlir
module {
  func @main() {
    %c1_i32 = arith.constant 1 : i32
    %c2_i32 = arith.constant 2 : i32
    %0 = arith.addi %c1_i32, %c2_i32 : i32
    return
  }
}

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$
```

图 50 基础 MLIR 程序的输出

再执行 canonicalize:

```
1 mlir-opt-14 test.mlir -canonicalize
```

由于常量和加法的结果都没有被使用，也没有可观察的副作用，规范化后函数体只保留 return。图51记录了转换前后的对比。

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ echo "==== original ====="
mlir-opt-14 test.mlir

echo "==== canonicalize ====="
mlir-opt-14 test.mlir -canonicalize
==== original =====
module {
  func @main() {
    %c1_i32 = arith.constant 1 : i32
    %c2_i32 = arith.constant 2 : i32
    %0 = arith.addi %c1_i32, %c2_i32 : i32
    return
  }
}

==== canonicalize =====
module {
  func @main() {
    return
  }
}

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$
```

图 51 Canonicalization 前后的 MLIR 对比

(三) Linalg Dialect 中的矩阵乘法

矩阵乘法程序 mlir/matmul.mlir 定义了 2×4 的 A 、 4×3 的 B 和 2×3 的 C 。高层表示只使用一条 linalg.matmul 操作，矩阵维度以及输入、输出 memref 类型都写在该操作中。

```
1 module {
2   func @matmul(%A: memref<2x4xf32>,
3               %B: memref<4x3xf32>,
```

```

4      %C: memref<2x3xf32>) {
5      linalg.matmul
6      ins(%A, %B : memref<2x4xf32>, memref<4x3xf32>)
7      outs(%C : memref<2x3xf32>)
8      return
9  }
10 }
```

代码 7 Linalg Dialect 中的矩阵乘法程序 matmul.mlir

```
1 mlir-opt-14 matmul.mlir
```

图52显示了高层 `linalg.matmul` 表示。此时可以直接看出矩阵乘法的输入、输出和维度，循环、地址计算等细节还没有展开。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 matmul.mlir
module {
  func @matmul(%arg0: memref<2x4xf32>, %arg1: memref<4x3xf32>, %arg2: memr
ef<2x3xf32>) {
    linalg.matmul ins(%arg0, %arg1 : memref<2x4xf32>, memref<4x3xf32>) out
s(%arg2 : memref<2x3xf32>)
    return
  }
}

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$
```

图 52 Linalg Dialect 中的矩阵乘法表示

(四) Linalg 到 SCF、MemRef 和 Arith 的 Lowering

使用 `convert-linalg-to-loops` 后，`linalg.matmul` 展开为三层 `scf.for`。循环体使用 `memref.load` 读取 $A[i][k]$ 、 $B[k][j]$ 和 $C[i][j]$ ，使用 `arith.mulf`、`arith.addf` 完成乘法，再由 `memref.store` 写回 $C[i][j]$ 。

```

1 mlir-opt-14 matmul.mlir -convert-linalg-to-loops
2 mlir-opt-14 matmul.mlir -convert-linalg-to-loops -o matmul_loops.mlir
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 matmul.mlir -c
onvert-linalg-to-loops
module {
  func @matmul(%arg0: memref<2x4xf32>, %arg1: memref<4x3xf32>, %arg2: memr
ef<2x3xf32>) {
    %c2 = arith.constant 2 : index
    %c4 = arith.constant 4 : index
    %c3 = arith.constant 3 : index
    %c0 = arith.constant 0 : index
    %c1 = arith.constant 1 : index
    scf.for %arg3 = %c0 to %c2 step %c1 {
      scf.for %arg4 = %c0 to %c3 step %c1 {
        scf.for %arg5 = %c0 to %c4 step %c1 {
          %0 = memref.load %arg0[%arg3, %arg5] : memref<2x4xf32>
          %1 = memref.load %arg1[%arg5, %arg4] : memref<4x3xf32>
          %2 = memref.load %arg2[%arg3, %arg4] : memref<2x3xf32>
          %3 = arith.mulf %0, %1 : f32
          %4 = arith.addf %2, %3 : f32
          memref.store %4, %arg2[%arg3, %arg4] : memref<2x3xf32>
        }
      }
    }
    return
  }
}
```

```
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$
```

图 53 linalg.matmul Lowering 为三层 SCF 循环

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 matmul.mlir \
-convert-linalg-to-loops \
-o matmul_loops.mlir
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ ls -lh matmul.mlir matmul_
loops.mlir
-rwxrwxrwx 1 lenovo lenovo 248 Sep 26 17:32 matmul.mlir
-rwxrwxrwx 1 lenovo lenovo 785 Sep 26 18:50 matmul_loops.mlir
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ cat matmul_loops.mlir
module {
  func @matmul(%arg0: memref<2x4xf32>, %arg1: memref<4x3xf32>, %arg2: memr
ef<2x3xf32>) {
    %c2 = arith.constant 2 : index
    %c4 = arith.constant 4 : index
    %c3 = arith.constant 3 : index
    %c0 = arith.constant 0 : index
    %c1 = arith.constant 1 : index
    scf.for %arg3 = %c0 to %c2 step %c1 {
      scf.for %arg4 = %c0 to %c3 step %c1 {
        scf.for %arg5 = %c0 to %c4 step %c1 {
          %0 = memref.load %arg0[%arg3, %arg5] : memref<2x4xf32>
          %1 = memref.load %arg1[%arg5, %arg4] : memref<4x3xf32>
          %2 = memref.load %arg2[%arg3, %arg4] : memref<2x3xf32>
          %3 = arith.mulf %0, %1 : f32
          %4 = arith.addf %2, %3 : f32
          memref.store %4, %arg2[%arg3, %arg4] : memref<2x3xf32>
        }
      }
    }
    return
  }
}

```

图 54 matmul_loops.mlir 的保存结果

在 matmul_loops.mlir 中，最外层循环范围为 0 到 2，中间层为 0 到 3，最内层为 0 到 4。因此，矩阵乘法原来的单条高层操作已经变成了由循环边界、显式内存访问和浮点乘加组成的程序。

（五） SCF 到控制流形式的 Lowering

MLIR 14 的 Pass 名称与其他版本存在差异。先通过帮助信息确认本环境中可用的转换 Pass，再使用 convert-scf-to-std 处理保存的循环程序。

```

1 mlir-opt-14 --help | grep -E 'convert-(scf|memref|arith|std)'
2 mlir-opt-14 matmul_loops.mlir -convert-scf-to-std -o matmul_cfg.mlir

```

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 --help | grep
-E "convert-(scf|memref|arith|std).*llvm|convert-scf|convert-memref|conver
t-arith|convert-std"
    --convert-arith-to-llvm          - Convert Arithm
etic dialect to LLVM dialect
    --convert-arith-to-spirv        - Convert Arithm
etic dialect to SPIR-V dialect
    --convert-memref-to-llvm        - Convert operat
ions from the MemRef dialect to the LLVM dialect
    --convert-memref-to-spirv       - Convert MemRef
dialect to SPIR-V dialect
    --convert-scf-to-openmp         - Convert SCF pa
rallel loop to OpenMP parallel + workshare constructs.
    --convert-scf-to-spirv          - Convert SCF di
alect to SPIR-V dialect.
    --convert-scf-to-std             - Convert SCF di
alect to Standard dialect, replacing structured control flow with a CFG
    --convert-std-to-llvm           - Convert scalar
and vector operations from the Standard to the LLVM dialect
    --convert-std-to-spirv          - Convert Standa
rd dialect to SPIR-V dialect
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$

```

图 55 MLIR 14 中可用的关键 Lowering Pass

matmul_cfg.mlir 中不再出现 scf.for。三层循环被展开为基本块、br、cond_br 和块参数；比较指令 arith.cmpi 控制循环是否继续。循环体中的 memref.load、arith.mulf、arith.addf 和 memref.store 仍保留在基本块内。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt-14 matmul_loops.mlir \
-convert-scf-to-std \
-o matmul_cfg.mlir
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ cat matmul_cfg.mlir
module {
  func @matmul(%arg0: memref<2x4xf32>, %arg1: memref<4x3xf32>, %arg2: memref<2x3xf32>) {
    %c2 = arith.constant 2 : index
    %c4 = arith.constant 4 : index
    %c3 = arith.constant 3 : index
    %c0 = arith.constant 0 : index
    %c1 = arith.constant 1 : index
    br ^bb1(%c0 : index)
  ^bb1(%0: index): // 2 preds: ^bb0, ^bb8
    %1 = arith.cmpi slt, %0, %c2 : index
    cond_br %1, ^bb2, ^bb9
  ^bb2: // pred: ^bb1
    br ^bb3(%c0 : index)
  ^bb3(%2: index): // 2 preds: ^bb2, ^bb7
    %3 = arith.cmpi slt, %2, %c3 : index
    cond_br %3, ^bb4, ^bb8
  ^bb4: // pred: ^bb3
    br ^bb5(%c0 : index)
  ^bb5(%4: index): // 2 preds: ^bb4, ^bb6
    %5 = arith.cmpi slt, %4, %c4 : index
    cond_br %5, ^bb6, ^bb7
  ^bb6: // pred: ^bb5
    %6 = memref.load %arg0[%0, %4] : memref<2x4xf32>
    %7 = memref.load %arg1[%4, %2] : memref<4x3xf32>
    %8 = memref.load %arg2[%0, %2] : memref<2x3xf32>
    %9 = arith.mulf %6, %7 : f32
    %10 = arith.addf %8, %9 : f32
    memref.store %10, %arg2[%0, %2] : memref<2x3xf32>
    %11 = arith.addi %4, %c1 : index
    br ^bb5(%11 : index)
  ^bb7: // pred: ^bb5
    %12 = arith.addi %2, %c1 : index
    br ^bb3(%12 : index)
  ^bb8: // pred: ^bb3
    %13 = arith.addi %0, %c1 : index
    br ^bb1(%13 : index)
  ^bb9: // pred: ^bb1
    return
  }
}
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$

```

图 56 SCF 结构化循环转为基本块和条件分支

（六） 转换为 LLVM Dialect

在控制流已经展开后，继续将 Arith、MemRef 和 Standard 操作转换为 LLVM Dialect，并处理转换过程中出现的临时类型桥接：

```

1 mlir-opt-14 matmul_cfg.mlir \
2   -convert-arith-to-llvm -convert-memref-to-llvm \
3   -convert-std-to-llvm -reconcile-unrealized-casts \
4   -o matmul_llvm.mlir

```

转换结果中可以看到 `llvm.func`、`llvm.br`、`llvm.cond_br`、`llvm.getelementptr`、`llvm.load`、`llvm.fmul`、`llvm.fadd` 和 `llvm.store`。其中，`memref` 的访问已经被表示为指针、偏移和维度信息，矩阵元素地址由 `llvm.getelementptr` 计算。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ mlir-opt -14 matmul_cfg.mlir \
-convert-arith-to-llvm \
-convert-memref-to-llvm \
-convert-std-to-llvm \
-reconcile-unrealized-casts \
-o matmul_llvm.mlir
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ cat matmul_llvm.mlir
module attributes {llvm.data.layout = ""} {
  llvm.func @matmul(%arg0: !llvm.ptr<f32>, %arg1: !llvm.ptr<f32>, %arg2: i64, %arg3: i64, %arg4: i64, %arg5: i64, %arg6: i64, %arg7: !llvm.ptr<f32>, %arg8: !llvm.ptr<f32>, %arg9: i64, %arg10:
i64, %arg11: i64, %arg12: i64, %arg13: i64, %arg14: !llvm.ptr<f32>, %arg15: !llvm.ptr<f32>, %arg16: i64, %arg17: i64, %arg18: i64, %arg19: i64, %arg20: i64) {
  %0 = llvm.mlir.undef : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %1 = llvm.insertvalue %arg0, %0[0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %2 = llvm.insertvalue %arg1, %1[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %3 = llvm.insertvalue %arg2, %2[2] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %4 = llvm.insertvalue %arg3, %3[3, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %5 = llvm.insertvalue %arg4, %4[4, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %6 = llvm.insertvalue %arg5, %5[5, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %7 = llvm.insertvalue %arg6, %6[6, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %8 = llvm.mlir.undef : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %9 = llvm.insertvalue %arg7, %8[0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %10 = llvm.insertvalue %arg8, %9[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %11 = llvm.insertvalue %arg9, %10[2] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %12 = llvm.insertvalue %arg10, %11[3, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %13 = llvm.insertvalue %arg11, %12[4, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %14 = llvm.insertvalue %arg12, %13[5, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %15 = llvm.insertvalue %arg13, %14[6, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %16 = llvm.mlir.undef : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %17 = llvm.insertvalue %arg14, %16[0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %18 = llvm.insertvalue %arg15, %17[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %19 = llvm.insertvalue %arg16, %18[2] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %20 = llvm.insertvalue %arg17, %19[3, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %21 = llvm.insertvalue %arg18, %20[4, 0] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %22 = llvm.insertvalue %arg19, %21[5, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %23 = llvm.insertvalue %arg20, %22[6, 1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %24 = llvm.mlir.constant(2 : index) : i64
  %25 = llvm.mlir.constant(4 : index) : i64
  %26 = llvm.mlir.constant(3 : index) : i64
  %27 = llvm.mlir.constant(0 : index) : i64
  %28 = llvm.mlir.constant(1 : index) : i64
  llvmb1: // pred: ^bb0
  ^bb1(%29: i64): // 2 preds: ^bb0, ^bb8
  %30 = llvm.icmp "slt" %29, %24 : i64
  llvm.cond_br %30, ^bb2, ^bb9
  ^bb2: // pred: ^bb1
  llvmb3: // pred: ^bb3(%27 : i64)
  ^bb3(%31: i64): // 2 preds: ^bb2, ^bb7
  %32 = llvm.icmp "slt" %31, %26 : i64
  llvm.cond_br %32, ^bb4, ^bb8
  ^bb4: // pred: ^bb3
  llvmb5: // pred: ^bb5(%27 : i64)
  ^bb5(%33: i64): // 2 preds: ^bb4, ^bb6

```

图 57 矩阵乘法转换为 LLVM Dialect 的前半部分

```

  %34 = llvm.icmp "slt" %33, %25 : i64
  llvm.cond_br %34, ^bb6, ^bb7
  ^bb6: // pred: ^bb5
  %35 = llvm.extractvalue %7[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %36 = llvm.mlir.constant(4 : index) : i64
  %37 = llvm.mul %29, %36 : i64
  %38 = llvm.add %37, %33 : i64
  %39 = llvm.getelementptr %35[%38] : (!llvm.ptr<f32>, i64) -> !llvm.ptr<f32>
  %40 = llvm.load %39 : !llvm.ptr<f32>
  %41 = llvm.extractvalue %15[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %42 = llvm.mlir.constant(3 : index) : i64
  %43 = llvm.mul %33, %42 : i64
  %44 = llvm.add %43, %31 : i64
  %45 = llvm.getelementptr %41[%44] : (!llvm.ptr<f32>, i64) -> !llvm.ptr<f32>
  %46 = llvm.load %45 : !llvm.ptr<f32>
  %47 = llvm.extractvalue %23[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %48 = llvm.mlir.constant(3 : index) : i64
  %49 = llvm.mul %29, %48 : i64
  %50 = llvm.add %49, %31 : i64
  %51 = llvm.getelementptr %47[%50] : (!llvm.ptr<f32>, i64) -> !llvm.ptr<f32>
  %52 = llvm.load %51 : !llvm.ptr<f32>
  %53 = llvm.fmul %40, %46 : f32
  %54 = llvm.fadd %52, %53 : f32
  %55 = llvm.extractvalue %23[1] : !llvm.struct<(ptr<f32>, ptr<f32>, i64, array<2 x i64>, array<2 x i64>)>
  %56 = llvm.mlir.constant(3 : index) : i64
  %57 = llvm.mul %29, %56 : i64
  %58 = llvm.add %57, %31 : i64
  %59 = llvm.getelementptr %55[%58] : (!llvm.ptr<f32>, i64) -> !llvm.ptr<f32>
  llvm.store %54, %59 : !llvm.ptr<f32>
  %60 = llvm.add %33, %28 : i64
  llvmb7: // pred: ^bb5(%60 : i64)
  ^bb7: // pred: ^bb5
  %61 = llvm.add %31, %28 : i64
  llvmb8: // pred: ^bb3(%61 : i64)
  ^bb8: // pred: ^bb3
  %62 = llvm.add %29, %28 : i64
  llvmb9: // pred: ^bb1(%62 : i64)
  ^bb9: // pred: ^bb1
  llvm.return
}

```

图 58 LLVM Dialect 中的地址计算、读写和浮点乘加

(七) 标准 LLVM IR 与目标文件验证

matmul_llvm.mlir 仍属于 MLIR 体系中的 LLVM Dialect。使用 mlir-translate-14 将其生成标准 LLVM IR:

```
1 mlir-translate-14 --mlir-to-llvmir matmul_llvm.mlir -o matmul.ll
```

生成的 matmul.ll 中可以看到 define、phi、icmp、br、getelementptr、load、fmul、fadd、store 和 ret。循环变量通过 phi 在基本块之间传递，矩阵下标通过乘法和加法变成地址偏移。

```
lenovo@LAPTOP-G5T88NH0:/mnt/d/compilerlab/mlir$ mlir-translate-14 --mlir-to-llvmir matmul_llvm.mlir -o matmul.ll
lenovo@LAPTOP-G5T88NH0:/mnt/d/compilerlab/mlir$ ls -lh matmul_llvm.mlir matmul.ll
-rwxrwxrwx 1 lenovo lenovo 8.2K Sep 26 18:59 matmul.ll
-rwxrwxrwx 1 lenovo lenovo 5.5K Sep 26 18:57 matmul_llvm.mlir
lenovo@LAPTOP-G5T88NH0:/mnt/d/compilerlab/mlir$ head -n 80 matmul.ll
; ModuleID = "LLVMDialectModule"
source_filename = "LLVMDialectModule"

declare i8* @malloc(i64)

declare void @free(i8*)

define void @matmul(float* %0, float* %1, i64 %2, i64 %3, i64 %4, i64 %5, i64 %6, float* %7, float* %8, i64 %9, i64 %10, i64 %11, i64 %12, i64 %13, float* %14, float* %15, i64 %16, i64 %17, i64 %18, i64 %19, i64 %20) idbg {
  %22 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } undef, float* %0, idbg 17
  %23 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %22, float* %1, idbg 19
  %24 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %23, i64 %2, idbg 18
  %25 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %24, i64 %3, idbg 11
  %26 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %25, i64 %4, idbg 12
  %27 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %26, i64 %5, idbg 113
  %28 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %27, i64 %6, idbg 14
  %29 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } undef, float* %7, idbg 115
  %30 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %29, float* %8, idbg 16
  %31 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %30, i64 %9, idbg 17
  %32 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %31, i64 %10, idbg 118
  %33 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %32, i64 %11, idbg 119
  %34 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %33, i64 %12, idbg 120
  %35 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %34, i64 %13, idbg 121
  %36 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } undef, float* %14, idbg 122
  %37 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %36, float* %15, idbg 123
  %38 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %37, i64 %16, idbg 124
  %39 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %38, i64 %17, idbg 125
  %40 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %39, i64 %18, idbg 126
  %41 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %40, i64 %19, idbg 127
  %42 = insertvalue { float*, float*, i64, [2 x i64], [2 x i64] } %41, i64 %20, idbg 128
  br label %43, idbg 129

43:                                     ; preds = %79, %21
  %44 = phi i64 [ %80, %79 ], [ 0, %21 ]
  %45 = icmp slt i64 %44, 2, idbg 130
  br i1 %45, label %46, label %81, idbg 131

46:                                     ; preds = %43
  br label %47, idbg 132

47:                                     ; preds = %77, %46
  %48 = phi i64 [ %78, %77 ], [ 0, %46 ]
  %49 = icmp slt i64 %48, 3, idbg 133
  br i1 %49, label %50, label %79, idbg 134
```

图 59 MLIR LLVM Dialect 生成的标准 LLVM IR 前半部分

```

50:                                     ; preds = %47
  br label %51, !dbg !35

51:                                     ; preds = %54, %50
  %52 = phi i64 [ %76, %54 ], [ 0, %50 ]
  %53 = icmp slt i64 %52, 4, !dbg !36
  br i1 %53, label %54, label %77, !dbg !37

54:                                     ; preds = %51
  %55 = extractvalue { float*, float*, i64, [2 x i64], [2 x i64] } %28, 1, !dbg !38
  %56 = mul i64 %44, 4, !dbg !39
  %57 = add i64 %56, %52, !dbg !40
  %58 = getelementptr float, float* %55, i64 %57, !dbg !41
  %59 = load float, float* %58, align 4, !dbg !42
  %60 = extractvalue { float*, float*, i64, [2 x i64], [2 x i64] } %35, 1, !dbg !43
  %61 = mul i64 %52, 3, !dbg !44
  %62 = add i64 %61, %48, !dbg !45
  %63 = getelementptr float, float* %60, i64 %62, !dbg !46
  %64 = load float, float* %63, align 4, !dbg !47
  %65 = extractvalue { float*, float*, i64, [2 x i64], [2 x i64] } %42, 1, !dbg !48
  %66 = mul i64 %44, 3, !dbg !49
  %67 = add i64 %66, %48, !dbg !50
  %68 = getelementptr float, float* %65, i64 %67, !dbg !51
  %69 = load float, float* %68, align 4, !dbg !52
  %70 = fmul float %59, %64, !dbg !53
  %71 = fadd float %69, %70, !dbg !54
  %72 = extractvalue { float*, float*, i64, [2 x i64], [2 x i64] } %42, 1, !dbg !55
  %73 = mul i64 %44, 3, !dbg !56
  %74 = add i64 %73, %48, !dbg !57
  %75 = getelementptr float, float* %72, i64 %74, !dbg !58
  store float %71, float* %75, align 4, !dbg !59
  %76 = add i64 %52, 1, !dbg !60
  br label %51, !dbg !61

77:                                     ; preds = %51
  %78 = add i64 %48, 1, !dbg !62
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$

```

图 60 标准 LLVM IR 中的循环结构与浮点乘加

最后使用 Clang 将标准 LLVM IR 编译为目标文件：

```

1 clang -c matmul.ll -o matmul.o
2 file matmul.o
3 nm -g matmul.o | grep ' matmul$'

```

终端结果显示生成的 matmul.o 为 x86-64 的 ELF 可重定位文件，并且符号表中存在 matmul。本次检查再次使用 clang -c -O0 -g 编译仓库中的 matmul.ll，能够生成同样带有 matmul 符号和 .debug_info 节的目标文件。

```

lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ clang -c matmul.ll -o matmul.o
warning: overriding the module target triple with x86_64-pc-linux-gnu [-Woverride-module]
1 warning generated.
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ file matmul.o
matmul.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), with debug info, not stripped
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$ grep -E "define void @matmul|phi|icmp|br|getelementptr|load float|fmul|fadd|store float|ret void" matmul.ll | head -n 40
define void @matmul(float* %0, float* %1, i64 %2, i64 %3, i64 %4, i64 %5, i64 %6, float* %7, float* %8, i64 %9, i64 %10, i64 %11, i64 %12, i64 %13, float* %14, float* %15, i64 %16, i64 %17, i64 %18, i64 %19, i64 %20) !dbg !3 {
    br label %43, !dbg !29
    %44 = phi i64 [ %80, %79 ], [ 0, %21 ]
    %45 = icmp slt i64 %44, 2, !dbg !30
    br i1 %45, label %46, label %81, !dbg !31
    br label %47, !dbg !32
    %48 = phi i64 [ %78, %77 ], [ 0, %46 ]
    %49 = icmp slt i64 %48, 3, !dbg !33
    br i1 %49, label %50, label %79, !dbg !34
    br label %51, !dbg !35
    %52 = phi i64 [ %76, %54 ], [ 0, %50 ]
    %53 = icmp slt i64 %52, 4, !dbg !36
    br i1 %53, label %54, label %77, !dbg !37
    %58 = getelementptr float, float* %55, i64 %57, !dbg !41
    %59 = load float, float* %58, align 4, !dbg !42
    %63 = getelementptr float, float* %60, i64 %62, !dbg !46
    %64 = load float, float* %63, align 4, !dbg !47
    %68 = getelementptr float, float* %65, i64 %67, !dbg !51
    %69 = load float, float* %68, align 4, !dbg !52
    %70 = fmul float %59, %64, !dbg !53
    %71 = fadd float %69, %70, !dbg !54
    %75 = getelementptr float, float* %72, i64 %74, !dbg !58
    store float %71, float* %75, align 4, !dbg !59
    br label %51, !dbg !61
    br label %47, !dbg !63
    br label %43, !dbg !65
    ret void, !dbg !66
lenovo@LAPTOP-G5T08NH0:/mnt/d/compilerLab/mlir$

```

图 61 标准 LLVM IR 编译为 x86-64 可重定位目标文件

（八） 本部分小结

本部分从基础程序的 Canonicalization 开始,随后以矩阵乘法为例,依次观察 linalg.matmul、scf.for、基本块控制流、LLVM Dialect 和标准 LLVM IR。高层表示保存了矩阵维度和运算语义;经过 Lowering 后,循环边界、内存访问、地址计算和浮点乘加逐步变成更具体的操作。最终的 matmul.o 说明生成的 LLVM IR 已经能够交给 LLVM 后端生成 x86-64 目标文件。

六、 结论

小题 1 中, 成员 A 进行了 C++ 阶乘程序的 GCC 编译实验, 成员 B 进行了 C17 程序的 Clang/LLVM 编译实验。

成员 A 分析了 N、mul 和 DEBUG 的预处理结果, 并生成 factorial 对应的 tree、CFG、RTL、目标文件和链接结果。输入 5 时, 动态链接程序和静态链接程序均输出 120 和 1。将加入 -g 与未加入 -g 的目标文件进行对比, 加入 -g 后目标文件中包含调试相关段; 将使用 -fno-omit-frame-pointer 与默认方式生成的汇编进行对比, 前者保留了 %rbp 栈帧。

成员 B 分析了 MAX_ITEMS 和 ADJUST 的宏展开结果, 并生成 weighted_sum 的 CFG、LLVM IR、AArch64 汇编和目标文件, 同时检查符号表和重定位项。样例数组 1 2 3 4 5 的运行结果为 result=-11, processed=5。将未优化版本与 -O2 优化版本生成的代码进行对比, -O2 版本减少了部分重复的内存读写。

小题 2 中, 我们共同编写了读取数组、计算偶数和、统计正数个数的 SysY 程序, 并准备了三组基础测试数据。成员 A 进行了 LLVM IR 的编写和测试, 成员 B 进行了 RISC-V 汇编的编写和测试。

成员 A 编写的 LLVM IR 包含数组读写、循环、条件分支和运行时函数调用。我们分别使用 native 和 lli 运行三组测试数据, 两种运行方式的输出均为 12/4、6/2、0/5, 结果一致。

成员 B 在 RISC-V 汇编中使用 slli、add 和 lw 实现数组访问, 使用 remw/bnez 计算偶数和, 使用 blez 统计正数个数, 并在 main 中保存 ra 和 s0--s2。我们使用三组基础数据和 100 组扩展数据测试 RISC-V 程序, 100 组扩展测试全部通过, 失败数为 0。

我们对 LLVM IR 和 RISC-V 汇编的实现方式也进行了比较。LLVM IR 中主要涉及基本块跳转、phi 值和数组地址计算; RISC-V 汇编中还需要处理参数寄存器、返回值寄存器、保存寄存器、16 B 栈对齐和数组偏移。两部分程序都链接了 SysY 运行时库, 符号表中均可以找到 getint、putint 和 putch。

本次实验中, 我们实际接触了宏展开、CFG、LLVM IR、汇编代码、目标文件和链接结果, 也发现不同编译选项会直接影响生成代码和目标文件的内容。实验过程中, 我们进一步熟悉了 LLVM IR 中基本块、phi 指令和数组地址计算的写法, 也了解了 RISC-V 汇编中寄存器使用、栈对齐、函数调用和数组访问需要注意的细节。通过两部分实验, 我们对程序从源代码到可执行文件的编译过程, 以及 LLVM IR 和 RISC-V 汇编各自的特点有了更具体的理解。

进阶部分中, 成员 A 从 linalg.matmul 和三层 scf.for 对照矩阵乘法的循环结构, 成员 B 检查 br/cond_br、LLVM dialect 和 matmul.o 的符号。matmul.ll 重新编译后仍生成带 matmul 符号和 .debug_info 节的 x86-64 ELF 可重定位文件。通过基础部分和进阶部分的对照, 我们对程序从源代码到可执行文件的编译过程, 以及 LLVM IR、RISC-V 汇编和 MLIR 中端表示各自的特点有了更具体的理解。

参考文献

- [1] 编译原理课程组. 了解编译器、LLVM IR 编程及汇编编程实验指导 [课程资料].
- [2] 编译原理课程组. 编译器开发环境部署 [课程资料].
- [3] SysY 语言工作组. SysY 语言定义 (2022 版) [课程资料].
- [4] SysY 语言工作组. SysY 运行时库说明 [课程资料].

- [5] Free Software Foundation. *Using the GNU Compiler Collection (GCC)*. GCC Online Documentation.
- [6] LLVM Project. *LLVM Language Reference Manual*.
- [7] LLVM Project. *MLIR: Multi-Level Intermediate Representation Documentation*.
- [8] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*.

NIJUN